TypeScript + React 프론트엔드 개발 입문

1일 7시간 과정

자바/파이썬 개발자를 위한 실무 중심 TypeScript & React 입문

오늘의 일정

시간	세션	내용
09:30 – 10:00	Session 1	TypeScript 문법 기초 (강의, 30분)
10:00 – 10:30	Session 2	TypeScript 실습 — typescript-exercises (30분)
10:30 – 10:40	—	**휴식 (10분)**
10:40 – 11:40	Session 3	React + TypeScript 프로젝트 설정 (1시간)
11:40 – 13:00	—	**점심시간**
13:00 – 15:00	Session 4-5	React QuickStart — Tic-Tac-Toe & Thinking in React (2시간)
15:00 – 17:00	Session 6	종합실습: Social Dashboard 앱 구현 (2시간)

오늘의 목표

오늘의 목표

1. TypeScript 타입 시스템 이해

`string`, `number`, `interface`, `Union` 타입으로 **코드를 작성하기 전에 버그를 잡는** 방법을 배웁니다

오늘의 목표

1. TypeScript 타입 시스템 이해

`string`, `number`, `interface`, `Union` 타입으로 **코드를 작성하기 전에 버그를 잡는** 방법을 배웁니다

2. React 컴포넌트 개발 패턴 습득

`useState`, `props`, 이벤트 핸들러 — **함수형 컴포넌트**로 화면을 구성하는 React의 핵심 개념을 익힙니다

오늘의 목표

1. TypeScript 타입 시스템 이해

`string`, `number`, `interface`, `Union` 타입으로 **코드를 작성하기 전에 버그를 잡는** 방법을 배웁니다

2. React 컴포넌트 개발 패턴 습득

`useState`, `props`, 이벤트 핸들러 — **함수형 컴포넌트**로 화면을 구성하는 React의 핵심 개념을 익힙니다

3. Social Dashboard 앱 완성

오늘 배운 모든 것을 활용해서 **실제로 동작하는 웹 앱**을 직접 만들어 봅니다

Session 1

TypeScript 문법 기초

30분 | 강의

왜 TypeScript인가? (1/2)

JavaScript의 문제: 런타임까지 오류를 모릅니다

```
// JavaScript: 아무런 경고 없이 실행됩니다
function calculateTax(price, rate) {
  return price * rate;
}

// 정상 케이스: 1000원의 10% = 100
console.log(calculateTax(1000, 0.1));  // 100 ✓

// 실수 케이스: 문자열을 넘겨도 JavaScript는 경고하지 않습니다
console.log(calculateTax("1000", 0.1)); // 100 (암묵적 형변환!) ← 더 위험

// 최악의 케이스: 사용자가 결제 완료 후에야 버그를 발견합니다
function getUsername(user) {
  return user.name.toUpperCase(); // user가 null이면?
}
getUsername(null); // TypeError: 프로덕션 서버에서 터집니다
```

JavaScript는 **런타임**까지 오류를 알 수 없습니다 — 코드가 실행되어야 버그가 드러납니다

왜 TypeScript인가? (1/2)

JavaScript의 문제: 런타임까지 오류를 모릅니다

```
// JavaScript: 아무런 경고 없이 실행됩니다
function calculateTax(price, rate) {
  return price * rate;
}

// 정상 케이스: 1000원의 10% = 100
console.log(calculateTax(1000, 0.1));  // 100 ✓

// 실수 케이스: 문자열을 넘겨도 JavaScript는 경고하지 않습니다
console.log(calculateTax("1000", 0.1)); // 100 (암묵적 형변환!) ← 더 위험

// 최악의 케이스: 사용자가 결제 완료 후에야 버그를 발견합니다
function getUsername(user) {
  return user.name.toUpperCase(); // user가 null이면?
}
getUsername(null); // TypeError: 프로덕션 서버에서 터집니다
```

JavaScript는 **런타임**까지 오류를 알 수 없습니다 — 코드가 실행되어야 버그가 드러납니다

왜 TypeScript인가? (1/2)

JavaScript의 문제: 런타임까지 오류를 모릅니다

```
// JavaScript: 아무런 경고 없이 실행됩니다
function calculateTax(price, rate) {
  return price * rate;
}

// 정상 케이스: 1000원의 10% = 100
console.log(calculateTax(1000, 0.1));  // 100 ✓

// 실수 케이스: 문자열을 넘겨도 JavaScript는 경고하지 않습니다
console.log(calculateTax("1000", 0.1)); // 100 (암묵적 형변환!) ← 더 위험

// 최악의 케이스: 사용자가 결제 완료 후에야 버그를 발견합니다
function getUsername(user) {
  return user.name.toUpperCase(); // user가 null이면?
}
getUsername(null); // TypeError: 프로덕션 서버에서 터집니다
```

JavaScript는 **런타임**까지 오류를 알 수 없습니다 — 코드가 실행되어야 버그가 드러납니다

왜 TypeScript인가? (2/2)

TypeScript: 컴파일 타임에 오류를 잡아냅니다

```typescript
// TypeScript: 코드를 저장하는 순간 IDE가 빨간 줄로 표시합니다
function calculateTax(price: number, rate: number): number {
  return price * rate;
}

// 컴파일 오류 — 저장하자마자 IDE가 경고합니다
calculateTax("1000", 0.1);
// Argument of type 'string' is not assignable to parameter of type 'number'

// TypeScript = JavaScript + 타입 시스템
```

TypeScript 위치

```
TypeScript 코드 (.ts)
      ↓  tsc 컴파일
JavaScript 코드 (.js)
      ↓  브라우저/Node.js 실행
```

Java/C# 개발자에게: TypeScript의 타입 시스템은 정적 타입과 개념적으로 유사합니다. 다만 **구조적 타이핑 (Structural Typing)** 을 사용한다는 점이 다릅니다.

왜 TypeScript인가? (2/2)

TypeScript: 컴파일 타임에 오류를 잡아냅니다

```
function calculateTax(price: number, rate: number): number {
  return price * rate;
}

// 컴파일 오류 — 저장하자마자 IDE가 경고합니다
calculateTax("1000", 0.1);
// Argument of type 'string' is not assignable to parameter of type 'number'

// TypeScript = JavaScript + 타입 시스템
// JavaScript ⊂ TypeScript (JS는 TS의 부분집합)
// 모든 JS 코드는 TS에서도 동작합니다
```

TypeScript 위치

```
TypeScript 코드 (.ts)
      ↓  tsc 컴파일
JavaScript 코드 (.js)
      ↓  브라우저/Node.js 실행
```

Java/C# 개발자에게: TypeScript의 타입 시스템은 정적 타입과 개념적으로 유사합니다. 다만 **구조적 타이핑 (Structural Typing)** 을 사용한다는 점이 다릅니다.

왜 TypeScript인가? (2/2)

TypeScript: 컴파일 타임에 오류를 잡아냅니다

```
}

// 컴파일 오류 — 저장하자마자 IDE가 경고합니다
calculateTax("1000", 0.1);
// Argument of type 'string' is not assignable to parameter of type 'number'

// TypeScript = JavaScript + 타입 시스템
// JavaScript ⊂ TypeScript (JS는 TS의 부분집합)
// 모든 JS 코드는 TS에서도 동작합니다
// TS 코드는 최종적으로 JS로 컴파일됩니다
```

TypeScript 위치

```
TypeScript 코드 (.ts)
      ↓  tsc 컴파일
JavaScript 코드 (.js)
      ↓  브라우저/Node.js 실행
```

Java/C# 개발자에게: TypeScript의 타입 시스템은 정적 타입과 개념적으로 유사합니다. 다만 **구조적 타이핑 (Structural Typing)** 을 사용한다는 점이 다릅니다.

기본 타입 (1/3) — 원시 타입

```
// 올바른 예시: 반드시 소문자를 사용합니다
const userName: string = "홍길동";
const userAge: number = 28;
const isPremium: boolean = true;
const productPrice: number = 29.99; // 실수도 number
const nothing: null = null;

// 잘못된 예시: 절대 사용하지 마십시오
const userName2: String = "홍길동"; // String은 래퍼 객체
const userAge2: Number = 28;        // Number는 래퍼 객체
const isPremium2: Boolean = true;   // Boolean은 래퍼 객
// → VS Code에서 노란색 경고가 표시됩니다

// 배열 타입 — 두 가지 모두 동일합니다
const numbers: number[] = [1, 2, 3, 4, 5];
const names: string[] = ["홍길동", "김철수"];
const flags: Array<boolean> = [true, false, true]; //
```

흔한 실수: 대문자 타입

Java/C# 개발자가 가장 많이 하는 실수입니다.

올바른 사용	잘못된 사용
`string`	`String`
`number`	`Number`, `Integer`
`boolean`	`Boolean`

TypeScript에서 대문자 타입은 JavaScript의 **래퍼 객체**를 가리킵니다. 실무에서 `new String()` 을 쓸 이유는 없습니다.

기본 타입 (1/3) — 원시 타입

```
// 올바른 예시: 반드시 소문자를 사용합니다
const userName: string = "홍길동";
const userAge: number = 28;
const isPremium: boolean = true;
const productPrice: number = 29.99; // 실수도 number
const nothing: null = null;

// 잘못된 예시: 절대 사용하지 마십시오
const userName2: String = "홍길동"; // String은 래퍼 객체
const userAge2: Number = 28;         // Number는 래퍼 객체
const isPremium2: Boolean = true;    // Boolean은 래퍼 객체
// → VS Code에서 노란색 경고가 표시됩니다

// 배열 타입 — 두 가지 모두 동일합니다
const numbers: number[] = [1, 2, 3, 4, 5];
const names: string[] = ["홍길동", "김철수"];
const flags: Array<boolean> = [true, false, true]; // 
```

흔한 실수: 대문자 타입

Java/C# 개발자가 가장 많이 하는 실수입니다.

올바른 사용	잘못된 사용
`string`	`String`
`number`	`Number`, `Integer`
`boolean`	`Boolean`

TypeScript에서 대문자 타입은 JavaScript의 **래퍼 객체**를 가리킵니다. 실무에서 `new String()`을 쓸 이유는 없습니다.

기본 타입 (1/3) — 원시 타입

```
// 올바른 예시: 반드시 소문자를 사용합니다
const userName: string = "홍길동";
const userAge: number = 28;
const isPremium: boolean = true;
const productPrice: number = 29.99; // 실수도 number
const nothing: null = null;

// 잘못된 예시: 절대 사용하지 마십시오
const userName2: String = "홍길동"; // String은 래퍼 객체
const userAge2: Number = 28;        // Number는 래퍼 객체
const isPremium2: Boolean = true;   // Boolean은 래퍼 객
// → VS Code에서 노란색 경고가 표시됩니다

// 배열 타입 — 두 가지 모두 동일합니다
const numbers: number[] = [1, 2, 3, 4, 5];
const names: string[] = ["홍길동", "김철수"];
const flags: Array<boolean> = [true, false, true]; //
```

흔한 실수: 대문자 타입

Java/C# 개발자가 가장 많이 하는 실수입니다.

올바른 사용	잘못된 사용
`string`	`String`
`number`	`Number`, `Integer`
`boolean`	`Boolean`

TypeScript에서 대문자 타입은 JavaScript의 **래퍼 객체**를 가리킵니다. 실무에서 `new String()`을 쓸 이유는 없습니다.

기본 타입 (2/3) — 타입 추론과 any

```
// 타입 추론 — TypeScript가 오른쪽 값을 보고 타입을 자동으로 결정합니다
const productName = "노트북";    // TypeScript가 string으로 추론
const productCount = 10;         // TypeScript가 number로 추론
const inStock = true;            // TypeScript가 boolean으로 추론

// 위 코드는 아래와 완전히 동일합니다 — 굳이 쓸 필요 없습니다
const productName2: string = "노트북";  // 명시적 타입 (중복)
const productCount2: number = 10;       // 명시적 타입 (중복)

// 타입을 반드시 명시해야 하는 경우
let totalPrice: number;            // 초기값 없이 선언할 때
totalPrice = 50000;                // 정상
totalPrice = "오만원";             // 컴파일 오류!

const items: string[] = [];        // 빈 배열은 반드시 명시
```

`any` 를 쓰면 TypeScript 타입 검사가 전혀 이루어지지 않습니다. 타입을 모를 때는 `any` 대신 `unknown` 을 사용하십시오.

기본 타입 (2/3) — 타입 추론과 any

```
// 위 코드는 아래와 완전히 동일합니다 — 굳이 쓸 필요 없습니다
const productName2: string = "노트북";  // 명시적 타입 (중복)
const productCount2: number = 10;       // 명시적 타입 (중복)

// 타입을 반드시 명시해야 하는 경우
let totalPrice: number;            // 초기값 없이 선언할 때
totalPrice = 50000;                // 정상
totalPrice = "오만원";             // 컴파일 오류!

const items: string[] = [];        // 빈 배열은 반드시 명시
// const items2 = [];              // ← any[]로 추론됩니다 — 피하십시오

// 함수 매개변수는 추론이 되지 않으므로 항상 명시합니다
function greet(name: string): string {
  return `안녕하세요, ${name}님!`;
}
```

`any` 를 쓰면 TypeScript 타입 검사가 전혀 이루어지지 않습니다. 타입을 모를 때는 `any` 대신 `unknown` 을 사용하십시오.

기본 타입 (2/3) — 타입 추론과 any

```
totalPrice = 50000;                // 정상
totalPrice = "오만원";             // 컴파일 오류!

const items: string[] = [];        // 빈 배열은 반드시 명시
// const items2 = [];              // ← any[]로 추론됩니다 — 피하십시오

// 함수 매개변수는 추론이 되지 않으므로 항상 명시합니다
function greet(name: string): string {
  return `안녕하세요, ${name}님!`;
}

// any 타입 — TypeScript를 쓰는 이유를 없애버립니다
let data: any = "문자열";
data = 42;                     // 정상 — any이므로 뭐든 됩니다
data.nonExistentMethod();      // 컴파일 오류 없음 → 런타임에서 터집니다
```

`any` 를 쓰면 TypeScript 타입 검사가 전혀 이루어지지 않습니다. 타입을 모를 때는 `any` 대신 `unknown` 을 사용하십시오.

기본 타입 (3/3) — 객체와 인라인 타입

```
// 인라인 객체 타입 — 복잡해지면 읽기 어렵습니다
const user: { name: string; age: number; email: string } = {
  name: "홍길동",
  age: 28,
  email: "hong@example.com",
};

console.log(user.name);   // "홍길동" — 정상
console.log(user.phone);  // 컴파일 오류: 'phone' does not exist on type

// 해결책: interface나 type으로 이름을 붙여 재사용합니다 (다음 슬라이드에서 다룹니다)
// 지금은 이런 패턴이 있다는 것만 기억해 두십시오

// object 타입 — 피해야 합니다
const obj: object = { name: "홍길동" };
```

실무 팁: 객체 타입은 항상 `interface` 또는 `type` 으로 이름을 붙여서 사용합니다. 인라인으로 쓰면 재사용이 불가능하고 코드가 지저분해집니다.

기본 타입 (3/3) — 객체와 인라인 타입

```
console.log(user.name);    // "홍길동" — 정상
console.log(user.phone);   // 컴파일 오류: 'phone' does not exist on type

// 해결책: interface나 type으로 이름을 붙여 재사용합니다 (다음 슬라이드에서 다룹니다)
// 지금은 이런 패턴이 있다는 것만 기억해 두십시오

// object 타입 — 피해야 합니다
const obj: object = { name: "홍길동" };
obj.name; // 컴파일 오류: Property 'name' does not exist on type 'object'

// 빈 객체 타입 {} — 거의 아무것도 막지 않습니다
const anything: {} = 42;       // 정상 (의도치 않음)
const anything2: {} = "hello"; // 정상 (의도치 않음)
// → 이런 타입은 실무에서 사용하지 않습니다
```

실무 팁: 객체 타입은 항상 `interface` 또는 `type` 으로 이름을 붙여서 사용합니다. 인라인으로 쓰면 재사용이 불가능하고 코드가 지저분해집니다.

Interface와 Type (1/3)

interface — 객체 구조에 이름을 붙입니다

```
// interface 정의: 객체가 어떤 속성을 가져야 하는지 선언합니다
interface User {
  name: string;       // 필수 속성
  age: number;        // 필수 속성
  email: string;      // 필수 속성
  phone?: string;     // 선택적 속성 — ? 를 붙이면 없어도 됩니다
}

// interface를 타입으로 사용합니다
const user1: User = {
  name: "홍길동",
  age: 28,
  email: "hong@example.com",
  // phone 없어도 됩니다 — 선택적이므로
};
```

Interface와 Type (1/3)

interface — 객체 구조에 이름을 붙입니다

```
  phone?: string;    // 선택적 속성 — ? 를 붙이면 없어도 됩니다
}

// interface를 타입으로 사용합니다
const user1: User = {
  name: "홍길동",
  age: 28,
  email: "hong@example.com",
  // phone 없어도 됩니다 — 선택적이므로
};

const user2: User = {
  name: "김철수",
  age: 30,
  // email 없으면 컴파일 오류: Property 'email' is missing in type
};
```

Interface와 Type (1/3)

interface — 객체 구조에 이름을 붙입니다

```
};

const user2: User = {
  name: "김철수",
  age: 30,
  // email 없으면 컴파일 오류: Property 'email' is missing in type
};

// 함수 매개변수로 interface를 사용합니다
function displayUser(user: User): void {
  console.log(`${user.name} (${user.age}세) — ${user.email}`);
  if (user.phone) {
    // phone은 undefined일 수 있으므로 확인 후 사용합니다
    console.log(`연락처: ${user.phone}`);
  }
}
```

Interface와 Type (2/3)

type alias — interface보다 더 유연합니다

```
// type은 객체 타입뿐 아니라 어떤 타입에도 이름을 붙일 수 있습니다
type UserID = number;            // number에 별칭
type ProductName = string;       // string에 별칭
type ID = string | number;       // Union 타입 (다음 섹션에서 자세히 다룹니다)

// 객체 타입에 이름 붙이기 — interface와 유사합니다
type Point = { x: number; y: number };
const origin: Point = { x: 0, y: 0 };

// interface extends — 기존 인터페이스를 확장합니다
interface Animal {
  name: string;
}
interface Dog extends Animal {    // Animal의 모든 속성 + breed 추가
  breed: string;
```

Interface와 Type (2/3)

type alias — interface보다 더 유연합니다

```
// 객체 타입에 이름 붙이기 — interface와 유사합니다
type Point = { x: number; y: number };
const origin: Point = { x: 0, y: 0 };

// interface extends — 기존 인터페이스를 확장합니다
interface Animal {
  name: string;
}
interface Dog extends Animal {   // Animal의 모든 속성 + breed 추가
  breed: string;
}

// type 교차(intersection) — & 연산자로 합칩니다
type Animal2 = { name: string };
type Dog2 = Animal2 & { breed: string }; // 두 타입을 합칩니다
```

Interface와 Type (2/3)

type alias — interface보다 더 유연합니다

```
interface Dog extends Animal {   // Animal의 모든 속성 + breed 추가
  breed: string;
}

// type 교차(intersection) — & 연산자로 합칩니다
type Animal2 = { name: string };
type Dog2 = Animal2 & { breed: string }; // 두 타입을 합칩니다

// 두 방법의 결과는 동일합니다
const myDog1: Dog = { name: "멍이", breed: "진돗개" };
const myDog2: Dog2 = { name: "멍이", breed: "진돗개" };

// 실무에서 자주 쓰는 패턴: interface와 type 혼용
type Status = "active" | "inactive" | "pending"; // Union → type
interface UserProfile { name: string; status: Status; } // 객체 → interface
```

Interface와 Type (3/3)

언제 interface? 언제 type?

Interface와 Type (3/3)

언제 interface? 언제 type?

상황	권장	이유
객체 구조 정의	`interface`	확장(`extends`)이 직관적, 선언 병합 지원
유니온 타입	`type`	`interface` 로는 유니온 불가
조합/변형 타입	`type`	교차(`&`), 조건부 타입 등
React Props 정의	팀 컨벤션	둘 다 가능 — 일관성이 중요

Interface와 Type (3/3)

언제 interface? 언제 type?

상황	권장	이유
객체 구조 정의	`interface`	확장(`extends`)이 직관적, 선언 병합 지원
유니온 타입	`type`	`interface` 로는 유니온 불가
조합/변형 타입	`type`	교차(`&`), 조건부 타입 등
React Props 정의	팀 컨벤션	둘 다 가능 — 일관성이 중요

```
// 실무 예시: interface와 type을 적절히 혼용합니다
interface OrderItem {
  productId: number;
  quantity: number;
  unitPrice: number;
}

type PaymentMethod = "card" | "bank_transfer" | "kakao_pay";
```

Union & Literal 타입 (1/2)

Union 타입 — "A 또는 B"

```
// 기본 Union 타입: | 기호로 연결합니다
type StringOrNumber = string | number;

let value: StringOrNumber;
value = "안녕하세요"; // 정상 — string
value = 42;           // 정상 — number
value = true;         // 컴파일 오류 — boolean은 허용되지 않습니다

// API 응답 상태를 Literal Union으로 모델링합니다 — 실무에서 매우 자주 씁니다
type RequestStatus = "idle" | "loading" | "success" | "error";
//                   ↑ 정확히 이 4개의 문자열 값만 허용합니다

let status: RequestStatus;
```

Union & Literal 타입 (1/2)

Union 타입 — "A 또는 B"

```
// API 응답 상태를 Literal Union으로 모델링합니다 — 실무에서 매우 자주 씁니다
type RequestStatus = "idle" | "loading" | "success" | "error";
//                   ↑ 정확히 이 4개의 문자열 값만 허용합니다

let status: RequestStatus;
status = "loading";    // 정상
status = "success";    // 정상
status = "completed";  // 컴파일 오류: "completed"는 허용되지 않습니다
status = "Loading";    // 컴파일 오류: 대소문자 구분 주의!

// 실무 패턴: API 응답 전체를 타입으로 모델링합니다
interface ApiResponse<T> {
  status: RequestStatus;
```

Union & Literal 타입 (2/2)

타입 좁히기 (Type Narrowing) — Union 타입을 다루는 방법

```
// typeof로 원시 타입을 구분합니다
function processValue(value: string | number): string {
  if (typeof value === "string") {
    // 이 블록 안에서 TypeScript는 value가 string임을 압니다
    return value.toUpperCase(); // string 메서드 사용 가능
  }
  // 여기서는 TypeScript가 value가 number임을 압니다
  return value.toFixed(2); // number 메서드 사용 가능
}
console.log(processValue("hello")); // "HELLO"
console.log(processValue(3.14159)); // "3.14"
```

Union & Literal 타입 (2/2)

타입 좁히기 (Type Narrowing) — Union 타입을 다루는 방법

```
// in 연산자로 속성 존재를 확인합니다
interface Cat { name: string; meow(): void; }
interface Dog { name: string; bark(): void; }

function makeSound(animal: Cat | Dog): void {
  if ("meow" in animal) {
    animal.meow(); // Cat 타입으로 좁혀졌습니다
  } else {
    animal.bark(); // Dog 타입으로 좁혀졌습니다
  }
}
```

Union & Literal 타입 (2/2)

타입 좁히기 (Type Narrowing) — Union 타입을 다루는 방법

```
}

// 실무 예: API 상태에 따른 UI 분기
function handleResponse(response: ApiResponse<User>): void {
  if (response.status === "loading") {
    console.log("로딩 중...");
    return;
  }
  if (response.status === "error") {
    console.error(`오류: ${response.errorMessage}`);
    return;
  }
  if (response.status === "success" && response.data) {
```

제네릭 맛보기

`<T>` 패턴 — React에서 바로 사용합니다

```
// 제네릭 없이: 타입별로 함수를 각각 만들어야 합니다
function getFirstNumber(arr: number[]): number { return arr[0]; }
function getFirstString(arr: string[]): string { return arr[0]; }
// → 로직이 동일한데 타입만 다릅니다. 비효율적입니다.

// 제네릭으로 하나의 함수로 합칩니다
function getFirst<T>(arr: T[]): T {
  return arr[0]; // T 타입 배열의 첫 번째 요소를 T 타입으로 반환
}
// 사용할 때 T가 자동으로 결정됩니다
const firstNum = getFirst([1, 2, 3]);        // T = number
const firstStr = getFirst(["a", "b", "c"]); // T = string

// React에서 가장 먼저 만나는 제네릭: useState
```

지금 당장 완전히 이해 안 해도 됩니다. `useState<User[]>([])` 처럼 꺾쇠괄호 안에 타입을 적는 패턴을 기억해 두십시오. React 실습에서 반복하다 보면 자연스럽게 익힙니다.

제네릭 맛보기

`<T>` 패턴 — React에서 바로 사용합니다

```
}
// 사용할 때 T가 자동으로 결정됩니다
const firstNum = getFirst([1, 2, 3]);       // T = number
const firstStr = getFirst(["a", "b", "c"]); // T = string

// React에서 가장 먼저 만나는 제네릭: useState
import { useState } from "react";

const [count, setCount] = useState<number>(0);
setCount(1);       // 정상
setCount("hello"); // 컴파일 오류: string은 number에 할당 불가

const [user, setUser] = useState<User | null>(null);
// user는 User 타입이거나 null입니다
```

지금 당장 완전히 이해 안 해도 됩니다. `useState<User[]>([])` 처럼 꺾쇠괄호 안에 타입을 적는 패턴을 기억해 두십시오. React 실습에서 반복하다 보면 자연스럽게 익힙니다.

함수 타입

매개변수와 반환 타입 — React 이벤트 핸들러 예고

```
// 기본 함수 타입
function add(a: number, b: number): number {
  return a + b;
}

function logMessage(message: string): void { // void = 반환값 없음
  console.log(`[LOG] ${message}`);
}

const multiply = (a: number, b: number): number => a * b; // 화살표 함수도 동일

// 함수 타입을 타입으로 표현할 수 있습니다
type AddFunction = (a: number, b: number) => number;
const myAdd: AddFunction = (a, b) => a + b; // 타입이 이미 선언되어 있어 생략 가능
```

React에서 중요한 이유: 컴포넌트가 다른 컴포넌트에 함수를 prop으로 전달할 때, 함수 타입을 정확하게 맞춰야 합니다. Session 3에서 실제 React 코드와 함께 다시 설명합니다.

함수 타입

매개변수와 반환 타입 — React 이벤트 핸들러 예고

```
const multiply = (a: number, b: number): number => a * b; // 화살표 함수도 동일

// 함수 타입을 타입으로 표현할 수 있습니다
type AddFunction = (a: number, b: number) => number;
const myAdd: AddFunction = (a, b) => a + b; // 타입이 이미 선언되어 있어 생략 가능

// React 컴포넌트에서 이벤트 핸들러는 함수 타입이 필요합니다 (Session 3에서 자세히)
interface ButtonProps {
  label: string;
  onClick: () => void;                    // 매개변수 없고 반환값 없는 함수
  onChange?: (value: string) => void;     // 선택적 이벤트 핸들러
  onSubmit?: (event: React.FormEvent) => void;
}
```

React에서 중요한 이유: 컴포넌트가 다른 컴포넌트에 함수를 prop으로 전달할 때, 함수 타입을 정확하게 맞춰야 합니다. Session 3에서 실제 React 코드와 함께 다시 설명합니다.

Session 1 핵심 요약

Session 1 핵심 요약

개념	올바른 사용	주의사항
원시 타입	`string` `number` `boolean`	대문자 `String` `Number` 금지
배열	`number[]` 또는 `Array<number>`	빈 배열 `[]` 은 타입 명시 필수
`any`	사용하지 않습니다	대신 `unknown` 사용
객체 구조	`interface User { ... }`	인라인 타입 `{...}` 재사용 불가
조합 타입	`type Status = "a" | "b"`	유니온은 `type` 으로
타입 좁히기	`typeof` , `in` , Literal 비교	Union 사용 시 필수
제네릭	`useState<User[]>([])`	꺾쇠 안에 타입

Session 1 핵심 요약

개념	올바른 사용	주의사항
원시 타입	`string` `number` `boolean`	대문자 `String` `Number` 금지
배열	`number[]` 또는 `Array<number>`	빈 배열 `[]` 은 타입 명시 필수
`any`	사용하지 않습니다	대신 `unknown` 사용
객체 구조	`interface User { ... }`	인라인 타입 `{...}` 재사용 불가
조합 타입	`type Status = "a" | "b"`	유니온은 `type` 으로
타입 좁히기	`typeof` , `in` , Literal 비교	Union 사용 시 필수
제네릭	`useState<User[]>([])`	꺾쇠 안에 타입

Session 1 퀴즈

다음 중 올바른 TypeScript 코드는 무엇일까요?

Session 1 퀴즈

다음 중 올바른 TypeScript 코드는 무엇일까요?

```typescript
// (A)
const name: String = "홍길동";
const age: Integer = 28;
const isActive: Boolean = true;

// (B)
const name: string = "홍길동";
const age: number = 28;
const isActive: boolean = true;

// (C)
const name = "홍길동";      // 타입 명시 없음
const age = 28;
const isActive = true;
```

Session 1 퀴즈

다음 중 올바른 TypeScript 코드는 무엇일까요?

```
// (A)
const name: String = "홍길동";
const age: Integer = 28;
const isActive: Boolean = true;

// (B)
const name: string = "홍길동";
const age: number = 28;
const isActive: boolean = true;

// (C)
const name = "홍길동";      // 타입 명시 없음
const age = 28;
const isActive = true;
```

정답: (B)와 © 모두 올바른 코드입니다!

- **(A)**: `String`, `Integer`, `Boolean`은 TypeScript에서 사용하지 않습니다. Java/C# 습관입니다.
- **(B)**: 소문자 타입을 명시적으로 선언했습니다. 올바릅니다.

Session 2

TypeScript 실습

30분 | 독립 실습

https://typescript-exercises.github.io

실습 안내 — typescript-exercises.github.io

실습 안내 — typescript-exercises.github.io

진행 방법

1. 브라우저에서 **https://typescript-exercises.github.io** 접속
2. Exercise 1부터 순서대로 진행합니다
3. 왼쪽 편집기에 코드를 작성합니다
4. 오른쪽 패널에서 실시간 컴파일 결과를 확인합니다
5. 초록색 체크가 나오면 다음 Exercise로 넘어갑니다

실습 안내 — typescript-exercises.github.io

진행 방법

1. 브라우저에서 https://typescript-exercises.github.io 접속
2. Exercise 1부터 순서대로 진행합니다
3. 왼쪽 편집기에 코드를 작성합니다
4. 오른쪽 패널에서 실시간 컴파일 결과를 확인합니다
5. 초록색 체크가 나오면 다음 Exercise로 넘어갑니다

오늘 풀 문제: Exercise 1 ~ 4

Exercise	난이도	핵심 개념
Exercise 1	쉬움	`type` 정의, 기본 타입 (`string` , `number`)
Exercise 2	쉬움	`interface` , Union 타입 (`|`)
Exercise 3	보통	Literal 타입, Discriminated Union

Exercise 1-2 미리보기

Exercise 1: `type` 정의로 User 타입 완성하기

```
// TODO: User 타입에 name(string)과 age(number) 필드를 추가하십시오
export type User = {
    // 여기를 완성하세요
};
```

Exercise 1-2 미리보기

Exercise 1: `type` 정의로 User 타입 완성하기

```
// TODO: User 타입에 name(string)과 age(number) 필드를 추가하십시오
export type User = {
    // 여기를 완성하세요
};
```

Exercise 2: `interface` 와 Union 타입으로 Person 정의하기

```
// User와 Admin을 모두 포함하는 Person 타입을 정의하십시오
export interface Admin {
    // name, age, role 필드가 필요합니다
}

// Person은 User 또는 Admin 중 하나입니다
export type Person = unknown; // ← 이것을 수정하십시오

export function logPerson(person: Person) {
    console.log(` - ${person.name}, ${person.age}`); // ← 이것도 동작해야 합니다
}
```

Exercise 3-4 미리보기

Exercise 3: Discriminated Union + Literal 타입

```
// type 필드로 User와 Admin을 구분합니다 (Discriminated Union)
export interface User {
    type: 'user';        // 리터럴 타입: 정확히 'user' 문자열만 허용
    name: string;
    occupation: string;
}
export interface Admin {
    type: 'admin';       // 리터럴 타입: 정확히 'admin' 문자열만 허용
    name: string;
    role: string;
}
// logPerson 함수의 매개변수 타입을 User에서 Person으로 변경하면 됩니다
```

Exercise 3-4 미리보기

Exercise 3: Discriminated Union + Literal 타입

```
// type 필드로 User와 Admin을 구분합니다 (Discriminated Union)
export interface User {
    type: 'user';        // 리터럴 타입: 정확히 'user' 문자열만 허용
    name: string;
    occupation: string;
}
export interface Admin {
    type: 'admin';       // 리터럴 타입: 정확히 'admin' 문자열만 허용
    name: string;
    role: string;
}
// logPerson 함수의 매개변수 타입을 User에서 Person으로 변경하면 됩니다
```

Exercise 4: `in` 연산자와 타입 가드

```
// person이 Admin인지 확인하는 타입 가드 함수
export function isAdmin(person: Person): person is Admin {
    return 'role' in person; // Admin에만 role 속성이 있습니다
}
// 반환 타입 "person is Admin" — 이 함수가 true를 반환하면
// TypeScript가 해당 블록에서 person을 Admin으로 처리합니다
```

실습 팁 — 컴파일러 오류 읽는 법

실습 팁 — 컴파일러 오류 읽는 법

오류 메시지 구조 이해하기

```
Argument of type 'string' is not assignable to parameter of type 'number'.
↑                ↑                                          ↑
오류 종류        실제로 넘긴 타입                            기대했던 타입
```

실습 팁 — 컴파일러 오류 읽는 법

오류 메시지 구조 이해하기

```
Argument of type 'string' is not assignable to parameter of type 'number'.
↑                ↑                                          ↑
오류 종류        실제로 넘긴 타입                           기대했던 타입
```

자주 보는 오류와 해결법

오류 메시지	의미	해결 방법
`Property 'X' does not exist on type 'Y'`	Y 타입에 X 속성이 없음	interface에 X 속성 추가
`Type 'X' is not assignable to type 'Y'`	X를 Y에 할당 불가	타입을 일치시키거나 Union으로 확장
`Object literal may only specify known properties`	타입에 없는 속성을 가진 객체	타입 정의에 해당 속성 추가

실습 팁 — 컴파일러 오류 읽는 법

오류 메시지 구조 이해하기

```
Argument of type 'string' is not assignable to parameter of type 'number'.
↑                ↑                                          ↑
오류 종류        실제로 넘긴 타입                           기대했던 타입
```

자주 보는 오류와 해결법

오류 메시지	의미	해결 방법
`Property 'X' does not exist on type 'Y'`	Y 타입에 X 속성이 없음	interface에 X 속성 추가
`Type 'X' is not assignable to type 'Y'`	X를 Y에 할당 불가	타입을 일치시키거나 Union으로 확장
`Object literal may only specify known properties`	타입에 없는 속성을 가진 객체	타입 정의에 해당 속성 추가

Session 1-2 마무리

배운 개념 정리

Session 1-2 마무리

배운 개념 정리

```
// Session 1-2에서 배운 모든 개념이 여기 담겨 있습니다
interface User {                              // 객체 구조 → interface
  type: 'user';                              // Literal 타입
  name: string;                              // 원시 타입 (소문자!)
  age: number;
  occupation?: string;                        // 선택적 속성
}

interface Admin {
  type: 'admin';
  name: string;
  age: number;
  role: string;
}

type Person = User | Admin;                   // Union 타입 → type

function describe(person: Person): string { // 함수 타입
  if (person.type === 'user') {               // Discriminated Union
    return `${person.name}: ${person.occupation ?? '직업 미상'}`; // 타입 좁히기
  }
```

휴식 시간

10분 쉬어 갑니다

10:30 ~ 10:40

휴식 시간

10분 쉬어 갑니다

10:30 ~ 10:40

Session 3 준비사항

- 터미널(Terminal) 또는 명령 프롬프트를 열어 두십시오
- `node -v` 명령어로 Node.js가 설치되어 있는지 확인하십시오
- Node.js 18 이상이 필요합니다 (없으면 강사에게 알려 주십시오)

```
node -v     # v18.x.x 이상이어야 합니다
npm -v      # npm이 함께 설치되어 있습니다
```

Session 3

React + TypeScript 프로젝트 설정

React란 무엇인가

Meta(전 Facebook)가 만든 UI 라이브러리

핵심 아이디어: 컴포넌트(Component)

웹 페이지를 하나의 거대한 HTML 파일로 만드는 대신, 작은 독립적인 조각(컴포넌트)으로 나누어 만든 뒤 조합합니다.

React란 무엇인가

Meta(전 Facebook)가 만든 UI 라이브러리

핵심 아이디어: 컴포넌트(Component)

웹 페이지를 하나의 거대한 HTML 파일로 만드는 대신, 작은 독립적인 조각(컴포넌트)으로 나누어 만든 뒤 조합합니다.

레고 블록 비유

- 레고 블록 하나 = 컴포넌트 하나
- 각 블록은 독립적으로 만들어지고, 조합하여 전체 구조물을 완성
- 같은 블록을 여러 곳에서 재사용 가능

React란 무엇인가

Meta(전 Facebook)가 만든 UI 라이브러리

핵심 아이디어: 컴포넌트(Component)

웹 페이지를 하나의 거대한 HTML 파일로 만드는 대신, 작은 독립적인 조각(컴포넌트)으로 나누어 만든 뒤 조합합니다.

레고 블록 비유

- 레고 블록 하나 = 컴포넌트 하나
- 각 블록은 독립적으로 만들어지고, 조합하여 전체 구조물을 완성
- 같은 블록을 여러 곳에서 재사용 가능

중요: React는 프레임워크가 아닙니다

구분	예시	React의 위치

쇼핑몰 컴포넌트 트리

하나의 페이지를 컴포넌트로 분해하면?

```
<App>                          ← 최상위 컴포넌트
  <Header>
    <Logo />                   ← 로고 컴포넌트
    <SearchBar />              ← 검색창 컴포넌트
    <CartIcon />               ← 장바구니 아이콘 컴포넌트
  </Header>
  <ProductList>
    <ProductCard />            ← 상품 카드 (재사용!)
    <ProductCard />            ← 같은 컴포넌트, 다른 데이터
    <ProductCard />
  </ProductList>
  <Footer />
</App>
```

쇼핑몰 컴포넌트 트리

하나의 페이지를 컴포넌트로 분해하면?

```
<App>                          ← 최상위 컴포넌트
  <Header>
    <Logo />                   ← 로고 컴포넌트
    <SearchBar />              ← 검색창 컴포넌트
    <CartIcon />               ← 장바구니 아이콘 컴포넌트
  </Header>
  <ProductList>
    <ProductCard />            ← 상품 카드 (재사용!)
    <ProductCard />            ← 같은 컴포넌트, 다른 데이터
    <ProductCard />
  </ProductList>
  <Footer />
</App>
```

각 컴포넌트는 자신의 역할만 담당합니다

- `Header` : 상단 네비게이션 전체
- `ProductCard` : 상품 카드 하나의 UI + 로직

선언적 vs 명령적 프로그래밍

명령적 (Vanilla JS) "어떻게(How)" 만들지 단계별 지시

```
const h1 = document.createElement('h1');
h1.textContent = '안녕하세요, 김태민님!';
container.appendChild(h1);

const button = document.createElement('button');
const isLoggedIn = true;
if (isLoggedIn) {
  button.textContent = '로그아웃';
} else {
  button.textContent = '로그인';
}
container.appendChild(button);
```

선언적 (React) "무엇(What)"이 되어야 하는지 선언

```
function UserSection({ name, isLoggedIn }) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      <button>
        {isLoggedIn ? '로그아웃' : '로그인'}
      </button>
    </div>
  );
}
```

선언적 vs 명령적 프로그래밍

명령적 (Vanilla JS) "어떻게(How)" 만들지 단계별 지시

```
const h1 = document.createElement('h1');
h1.textContent = '안녕하세요, 김태민님!';
container.appendChild(h1);

const button = document.createElement('button');
const isLoggedIn = true;
if (isLoggedIn) {
  button.textContent = '로그아웃';
} else {
  button.textContent = '로그인';
}
container.appendChild(button);
```

선언적 (React) "무엇(What)"이 되어야 하는지 선언

```
function UserSection({ name, isLoggedIn }) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      <button>
        {isLoggedIn ? '로그아웃' : '로그인'}
      </button>
    </div>
  );
}
```

핵심 차이: 데이터(`isLoggedIn`)가 바뀌면 React가 DOM 업데이트를 자동으로 처리합니다. 개발자는 "이 상태일 때 화면이 이렇게 보여야 한다"만 선언하면 됩니다.

Vite로 프로젝트 생성

명령어 실행

```
npm create vite@latest my-react-app -- --template react-ts
```

Vite로 프로젝트 생성

명령어 실행

```
npm create vite@latest my-react-app -- --template react-ts
```

각 부분의 의미:

부분	의미
`npm create`	`npm init` 의 alias. create-* 패키지로 프로젝트 초기화
`vite@latest`	create-vite 패키지 최신 버전 사용
`my-react-app`	생성할 프로젝트 폴더 이름
`--`	이후 인자를 create-vite에 전달
`--template react-ts`	React + TypeScript 템플릿 선택

Vite로 프로젝트 생성

명령어 실행

```
npm create vite@latest my-react-app -- --template react-ts
```

각 부분의 의미:

부분	의미
`npm create`	`npm init` 의 alias. create-* 패키지로 프로젝트 초기화
`vite@latest`	create-vite 패키지 최신 버전 사용
`my-react-app`	생성할 프로젝트 폴더 이름
`--`	이후 인자를 create-vite에 전달
`--template react-ts`	React + TypeScript 템플릿 선택

프로젝트 설치 및 실행

```
cd my-react-app
npm install
npm run dev
```

프로젝트 설치 및 실행

```
cd my-react-app
npm install
npm run dev
```

`npm install` — 무슨 일이 일어나는가:

```
added 187 packages, and audited 188 packages in 8s
found 0 vulnerabilities
```

`package.json` 에 적힌 모든 의존성을 `node_modules/` 에 설치합니다. 187개라고 놀라지 마세요. React 하나를 설치해도 React가 필요로 하는 패키지들이 줄줄이 함께 설치됩니다.

프로젝트 설치 및 실행

```
cd my-react-app
npm install
npm run dev
```

`npm install` **— 무슨 일이 일어나는가:**

```
added 187 packages, and audited 188 packages in 8s
found 0 vulnerabilities
```

`package.json` 에 적힌 모든 의존성을 `node_modules/` 에 설치합니다. 187개라고 놀라지 마세요. React 하나를 설치해도 React가 필요로 하는 패키지들이 줄줄이 함께 설치됩니다.

`npm run dev` **— 개발 서버 시작:**

```
  VITE v5.x.x  ready in 342 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

브라우저에서 `http://localhost:5173` 접속!

프로젝트 설치 및 실행

```
cd my-react-app
npm install
npm run dev
```

`npm install` **— 무슨 일이 일어나는가:**

```
added 187 packages, and audited 188 packages in 8s
found 0 vulnerabilities
```

`package.json` 에 적힌 모든 의존성을 `node_modules/` 에 설치합니다. 187개라고 놀라지 마세요. React 하나를 설치해도 React가 필요로 하는 패키지들이 줄줄이 함께 설치됩니다.

`npm run dev` **— 개발 서버 시작:**

```
  VITE v5.x.x  ready in 342 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

브라우저에서 `http://localhost:5173` 접속!

프로젝트 구조 전체 그림

```
my-react-app/
├── node_modules/          # 설치된 npm 패키지 (git에 포함 안 함)
├── public/
│   └── vite.svg           # 정적 자산 (URL로 직접 접근)
├── src/
│   ├── assets/
│   │   └── react.svg
│   ├── App.css            # App 컴포넌트 스타일
│   ├── App.tsx            # 루트 컴포넌트 ← 주로 여기서 작업
│   ├── index.css          # 전역 스타일
│   └── main.tsx           # React 앱 진입점
├── index.html             # SPA의 HTML 진입점
├── package.json           # 프로젝트 메타데이터 및 의존성
├── tsconfig.json          # TypeScript 컴파일러 설정
└── vite.config.ts         # Vite 빌드 도구 설정
```

프로젝트 구조 전체 그림

```
my-react-app/
├── node_modules/          # 설치된 npm 패키지 (git에 포함 안 함)
├── public/
│   └── vite.svg           # 정적 자산 (URL로 직접 접근)
├── src/
│   ├── assets/
│   │   └── react.svg
│   ├── App.css            # App 컴포넌트 스타일
│   ├── App.tsx            # 루트 컴포넌트 ← 주로 여기서 작업
│   ├── index.css          # 전역 스타일
│   └── main.tsx           # React 앱 진입점
├── index.html             # SPA의 HTML 진입점
├── package.json           # 프로젝트 메타데이터 및 의존성
├── tsconfig.json          # TypeScript 컴파일러 설정
└── vite.config.ts         # Vite 빌드 도구 설정
```

가장 자주 보는 파일:

- `src/App.tsx` — 대부분의 작업이 여기서 시작
- `src/main.tsx` — React 앱을 HTML에 연결하는 진입점

index.html → main.tsx → App.tsx 흐름

```html
<body>
  <div id="root"></div>   <!—— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

```typescript
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

`index.html` → `main.tsx` → `App.tsx` 흐름

1단계: `index.html` — 브라우저가 처음 로드하는 파일

```html
<body>
  <div id="root"></div>   <!—— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

```typescript
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

`index.html` → `main.tsx` → `App.tsx` 흐름

1단계: `index.html` — 브라우저가 처음 로드하는 파일

```html
<body>
  <div id="root"></div>   <!—— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

```typescript
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

`index.html` → `main.tsx` → `App.tsx` 흐름

1단계: `index.html` — 브라우저가 처음 로드하는 파일

```
<body>
  <div id="root"></div>   <!—— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

2단계: `src/main.tsx` — React를 DOM에 연결

```
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

`index.html` → `main.tsx` → `App.tsx` 흐름

1단계: `index.html` — 브라우저가 처음 로드하는 파일

```
<body>
  <div id="root"></div>   <!—— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

2단계: `src/main.tsx` — React를 DOM에 연결

```
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

`index.html` → `main.tsx` → `App.tsx` 흐름

1단계: `index.html` — 브라우저가 처음 로드하는 파일

```
<body>
  <div id="root"></div>   <!— React가 여기에 그려넣음 -->
  <script type="module" src="/src/main.tsx"></script>
</body>
```

2단계: `src/main.tsx` — React를 DOM에 연결

```
import ReactDOM from 'react-dom/client'
import App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root')!)
  .render(<App />)
// getElementById('root') → index.html의 <div id="root">를 찾아서
// 그 안에 <App />  컴포넌트를 그려넣음
```

3단계: `src/App.tsx` — 실제 화면 내용

```
function App() {
  return <h1>Hello React!</h1>  // 여기서부터 시작
```

`tsconfig.json` 핵심 옵션

```
{
  "compilerOptions": {
    "strict": true,           // 모든 엄격한 타입 검사 활성화 (권장)
    "jsx": "react-jsx",       // JSX → React 자동 변환 (import React 불필요)
    "noEmit": true,           // JS 파일 생성 안 함 (빌드는 Vite가 담당)
    "noUnusedLocals": true,   // 사용 안 한 변수 선언 시 오류
    "noUnusedParameters": true, // 사용 안 한 파라미터 선언 시 오류
    "target": "ES2020",       // 컴파일 대상 JavaScript 버전
    "lib": ["ES2020", "DOM"] // 사용 가능한 API (브라우저 DOM 포함)
  }
}
```

`tsconfig.json` 핵심 옵션

```
{
  "compilerOptions": {
    "strict": true,           // 모든 엄격한 타입 검사 활성화 (권장)
    "jsx": "react-jsx",       // JSX → React 자동 변환 (import React 불필요)
    "noEmit": true,           // JS 파일 생성 안 함 (빌드는 Vite가 담당)
    "noUnusedLocals": true,   // 사용 안 한 변수 선언 시 오류
    "noUnusedParameters": true, // 사용 안 한 파라미터 선언 시 오류
    "target": "ES2020",       // 컴파일 대상 JavaScript 버전
    "lib": ["ES2020", "DOM"] // 사용 가능한 API (브라우저 DOM 포함)
  }
}
```

`tsconfig.json` 핵심 옵션

```
{
  "compilerOptions": {
    "strict": true,           // 모든 엄격한 타입 검사 활성화 (권장)
    "jsx": "react-jsx",       // JSX → React 자동 변환 (import React 불필요)
    "noEmit": true,           // JS 파일 생성 안 함 (빌드는 Vite가 담당)
    "noUnusedLocals": true,  // 사용 안 한 변수 선언 시 오류
    "noUnusedParameters": true, // 사용 안 한 파라미터 선언 시 오류
    "target": "ES2020",       // 컴파일 대상 JavaScript 버전
    "lib": ["ES2020", "DOM"] // 사용 가능한 API (브라우저 DOM 포함)
  }
}
```

`tsconfig.json` 핵심 옵션

```
{
  "compilerOptions": {
    "strict": true,           // 모든 엄격한 타입 검사 활성화 (권장)
    "jsx": "react-jsx",       // JSX → React 자동 변환 (import React 불필요)
    "noEmit": true,           // JS 파일 생성 안 함 (빌드는 Vite가 담당)
    "noUnusedLocals": true,   // 사용 안 한 변수 선언 시 오류
    "noUnusedParameters": true, // 사용 안 한 파라미터 선언 시 오류
    "target": "ES2020",       // 컴파일 대상 JavaScript 버전
    "lib": ["ES2020", "DOM"] // 사용 가능한 API (브라우저 DOM 포함)
  }
}
```

`tsconfig.json` 핵심 옵션

```
{
  "compilerOptions": {
    "strict": true,           // 모든 엄격한 타입 검사 활성화 (권장)
    "jsx": "react-jsx",       // JSX → React 자동 변환 (import React 불필요)
    "noEmit": true,           // JS 파일 생성 안 함 (빌드는 Vite가 담당)
    "noUnusedLocals": true,   // 사용 안 한 변수 선언 시 오류
    "noUnusedParameters": true, // 사용 안 한 파라미터 선언 시 오류
    "target": "ES2020",       // 컴파일 대상 JavaScript 버전
    "lib": ["ES2020", "DOM"] // 사용 가능한 API (브라우저 DOM 포함)
  }
}
```

실무 팁: `"strict": true` 를 처음에 켜두면 오류가 많이 나와서 힘들 수 있습니다. 하지만 이 오류들이 나중에 생길 버그를 미리 잡아줍니다. 처음부터 strict 모드로 개발하는 것을 강력히 권장합니다.

첫 번째 컴포넌트: Greeting.tsx

`src/Greeting.tsx` 파일을 새로 만들어보겠습니다.

```
// src/Greeting.tsx

interface GreetingProps {
  name: string;
  role?: string;  // ? = 선택적 prop (없어도 오류 없음)
}

function Greeting({ name, role }: GreetingProps) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      {role && <p>역할: {role}</p>}
    </div>
  );
}

export default Greeting;
```

첫 번째 컴포넌트: Greeting.tsx

`src/Greeting.tsx` 파일을 새로 만들어보겠습니다.

```
// src/Greeting.tsx

interface GreetingProps {
  name: string;
  role?: string;  // ? = 선택적 prop (없어도 오류 없음)
}

function Greeting({ name, role }: GreetingProps) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      {role && <p>역할: {role}</p>}
    </div>
  );
}

export default Greeting;
```

첫 번째 컴포넌트: Greeting.tsx

`src/Greeting.tsx` 파일을 새로 만들어보겠습니다.

```
// src/Greeting.tsx

interface GreetingProps {
  name: string;
  role?: string;  // ? = 선택적 prop (없어도 오류 없음)
}

function Greeting({ name, role }: GreetingProps) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      {role && <p>역할: {role}</p>}
    </div>
  );
}

export default Greeting;
```

첫 번째 컴포넌트: Greeting.tsx

`src/Greeting.tsx` 파일을 새로 만들어보겠습니다.

```
// src/Greeting.tsx

interface GreetingProps {
  name: string;
  role?: string;  // ? = 선택적 prop (없어도 오류 없음)
}

function Greeting({ name, role }: GreetingProps) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      {role && <p>역할: {role}</p>}
    </div>
  );
}

export default Greeting;
```

첫 번째 컴포넌트: Greeting.tsx

`src/Greeting.tsx` 파일을 새로 만들어보겠습니다.

```
// src/Greeting.tsx

interface GreetingProps {
  name: string;
  role?: string;  // ? = 선택적 prop (없어도 오류 없음)
}

function Greeting({ name, role }: GreetingProps) {
  return (
    <div>
      <h1>안녕하세요, {name}님!</h1>
      {role && <p>역할: {role}</p>}
    </div>
  );
}

export default Greeting;
```

구조 분해 할당: `({ name, role }: GreetingProps)` = props를 분해해서 바로 변수로 사용. `props.name` 대신

App.tsx에서 Greeting 사용

TypeScript 자동완성의 위력을 체험해봅시다.

```
// src/App.tsx
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting name="김태민" role="수강생" />
      <Greeting name="이선생" role="강사" />
      <Greeting name="박철수" />   {/* role 없어도 OK */}
    </div>
  );
}

export default App;
```

App.tsx에서 Greeting 사용

TypeScript 자동완성의 위력을 체험해봅시다.

```
// src/App.tsx
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting name="김태민" role="수강생" />
      <Greeting name="이선생" role="강사" />
      <Greeting name="박철수" />   {/* role 없어도 OK */}
    </div>
  );
}

export default App;
```

App.tsx에서 Greeting 사용

TypeScript 자동완성의 위력을 체험해봅시다.

```
// src/App.tsx
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting name="김태민" role="수강생" />
      <Greeting name="이선생" role="강사" />
      <Greeting name="박철수" />   {/* role 없어도 OK */}
    </div>
  );
}

export default App;
```

App.tsx에서 Greeting 사용

TypeScript 자동완성의 위력을 체험해봅시다.

```
// src/App.tsx
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting name="김태민" role="수강생" />
      <Greeting name="이선생" role="강사" />
      <Greeting name="박철수" />   {/* role 없어도 OK */}
    </div>
  );
}

export default App;
```

TypeScript + React의 강점 체험:

1. `<Greeting` 입력 후 스페이스 → `name`, `role` 자동완성 제안
2. `name` prop을 빠뜨리면 → 빨간 줄 오류 즉시 표시

named export vs default export

```
// ── default export ──────────────────────────────
// 파일 하나에 하나만 가능. import 시 이름 자유롭게 변경 가능.
export default function Greeting() { ... }

import Greeting from './Greeting';     // 정상
import MyGreeting from './Greeting';   // 이것도 정상! (이름 변경 가능)


// ── named export ────────────────────────────────
// 파일 하나에 여러 개 가능. import 시 반드시 원래 이름 사용.
export function Greeting() { ... }
export function Farewell() { ... }
export const MAX_USERS = 100;

import { Greeting, Farewell } from './Greeting';      // 중괄호 필수
import { Greeting as Hello } from './Greeting';       // 별칭 사용 시 as
```

named export vs default export

```
// ── default export ──────────────────────────────
// 파일 하나에 하나만 가능. import 시 이름 자유롭게 변경 가능.
export default function Greeting() { ... }

import Greeting from './Greeting';     // 정상
import MyGreeting from './Greeting';   // 이것도 정상! (이름 변경 가능)


// ── named export ────────────────────────────────
// 파일 하나에 여러 개 가능. import 시 반드시 원래 이름 사용.
export function Greeting() { ... }
export function Farewell() { ... }
export const MAX_USERS = 100;

import { Greeting, Farewell } from './Greeting';      // 중괄호 필수
import { Greeting as Hello } from './Greeting';       // 별칭 사용 시 as
```

named export vs default export

```
// ── default export ──────────────────────────────
// 파일 하나에 하나만 가능. import 시 이름 자유롭게 변경 가능.
export default function Greeting() { ... }

import Greeting from './Greeting';     // 정상
import MyGreeting from './Greeting';  // 이것도 정상! (이름 변경 가능)


// ── named export ────────────────────────────────
// 파일 하나에 여러 개 가능. import 시 반드시 원래 이름 사용.
export function Greeting() { ... }
export function Farewell() { ... }
export const MAX_USERS = 100;

import { Greeting, Farewell } from './Greeting';      // 중괄호 필수
import { Greeting as Hello } from './Greeting';       // 별칭 사용 시 as
```

named export vs default export

```
// ── default export ──────────────────────────────
// 파일 하나에 하나만 가능. import 시 이름 자유롭게 변경 가능.
export default function Greeting() { ... }

import Greeting from './Greeting';     // 정상
import MyGreeting from './Greeting';   // 이것도 정상! (이름 변경 가능)


// ── named export ────────────────────────────────
// 파일 하나에 여러 개 가능. import 시 반드시 원래 이름 사용.
export function Greeting() { ... }
export function Farewell() { ... }
export const MAX_USERS = 100;

import { Greeting, Farewell } from './Greeting';      // 중괄호 필수
import { Greeting as Hello } from './Greeting';       // 별칭 사용 시 as
```

현업 관례: 컴포넌트 파일은 `export default`, 여러 유틸리티/상수는 `named export`. 가장 중요한 것은 **팀 내 일관성**입니다.

JSX 규칙 1-2: 루트 요소 + className

규칙 1: 반드시 하나의 최상위(루트) 요소

```
// 오류 — 루트 요소가 2개
function Wrong() {
  return (
    <h1>제목</h1>
    <p>본문</p>    // ❌
  );
}
```

```
// 해결 — Fragment로 감싸기
function Right() {
  return (
    <>
      <h1>제목</h1>
      <p>본문</p>
    </>
  );
}
```

JSX 규칙 1-2: 루트 요소 + className

규칙 1: 반드시 하나의 최상위(루트) 요소

```
// 오류 — 루트 요소가 2개
function Wrong() {
  return (
    <h1>제목</h1>
    <p>본문</p>    // ❌
  );
}
```

```
// 해결 — Fragment로 감싸기
function Right() {
  return (
    <>
      <h1>제목</h1>
      <p>본문</p>
    </>
  );
}
```

규칙 2: `class` 대신 `className`

```
// ❌ HTML처럼 class를 쓰면 오류
<div class="container">...</div>

// ✅ JSX에서는 반드시 className
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>       // 연산
<p>길이: {name.length}글자</p>      // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>      // 연산
<p>길이: {name.length}글자</p>     // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>      // 연산
<p>길이: {name.length}글자</p>     // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

규칙 4: 조건부 렌더링 패턴

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>      // 연산
<p>길이: {name.length}글자</p>     // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

규칙 4: 조건부 렌더링 패턴

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>      // 연산
<p>길이: {name.length}글자</p>     // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

규칙 4: 조건부 렌더링 패턴

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 3-4: 중괄호 표현식 + 조건부 렌더링

규칙 3: `{}` 로 JavaScript 표현식 삽입

```
const name = '김철수';
const age = 25;

// 변수, 연산, 함수 호출 모두 가능
<h1>{name}</h1>                    // 변수
<p>내년 나이: {age + 1}세</p>      // 연산
<p>길이: {name.length}글자</p>     // 메서드 호출
<span>{isOK ? '확인' : '취소'}</span>  // 삼항 연산자
```

규칙 4: 조건부 렌더링 패턴

```
// 패턴 1: && 연산자 (보여주거나 / 안 보여주거나)
{isLoggedIn && <p>환영합니다!</p>}

// 패턴 2: 삼항 연산자 (A 보여주거나 / B 보여주거나)
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// 패턴 3: if 문은 {} 안에 직접 못 씀 — 함수 밖에서 처리
if (!isVisible) return null;  // 컴포넌트 return 전에 처리
```

JSX 규칙 5: 리스트 렌더링 + key prop

`.map()` 으로 배열을 JSX로 변환

```
const students = [
  { id: 1, name: '김철수', grade: 'A' },
  { id: 2, name: '이영희', grade: 'B' },
  { id: 3, name: '박민준', grade: 'A' },
];

// ✅ key에 안정적인 ID 사용
function StudentList() {
  return (
    <ul>
      {students.map((student) => (
        <li key={student.id}>
          {student.name} — {student.grade}
        </li>
      ))}
    </ul>
  );
}
```

JSX 규칙 5: 리스트 렌더링 + key prop

`.map()` 으로 배열을 JSX로 변환

```
  { id: 1, name: '김철수', grade: 'A' },
  { id: 2, name: '이영희', grade: 'B' },
  { id: 3, name: '박민준', grade: 'A' },
];

// ✅ key에 안정적인 ID 사용
function StudentList() {
  return (
    <ul>
      {students.map((student) => (
        <li key={student.id}>
          {student.name} — {student.grade}
        </li>
      ))}
    </ul>
  );
}

// ❌ 인덱스를 key로 사용하면 안 됩니다 (항목 추가/삭제 시 버그 위험)
```

JSX 규칙 5: 리스트 렌더링 + key prop

`.map()` 으로 배열을 JSX로 변환

```
],

// ✅ key에 안정적인 ID 사용
function StudentList() {
  return (
    <ul>
      {students.map((student) => (
        <li key={student.id}>
          {student.name} — {student.grade}
        </li>
      ))}
    </ul>
  );
}

// ❌ 인덱스를 key로 사용하면 안 됩니다 (항목 추가/삭제 시 버그 위험)
{students.map((student, index) => (
  <li key={index}>{student.name}</li>  // 피해야 함
))}
```

JSX 규칙 5: 리스트 렌더링 + key prop

`.map()` 으로 배열을 JSX로 변환

```
],

// ✅ key에 안정적인 ID 사용
function StudentList() {
  return (
    <ul>
      {students.map((student) => (
        <li key={student.id}>
          {student.name} — {student.grade}
        </li>
      ))}
    </ul>
  );
}

// ❌ 인덱스를 key로 사용하면 안 됩니다 (항목 추가/삭제 시 버그 위험)
{students.map((student, index) => (
  <li key={index}>{student.name}</li>  // 피해야 함
))}
```

독립 실습: ProductCard 컴포넌트

10분 독립 실습 — `src/ProductCard.tsx` 파일을 새로 만들어보세요!

요구사항 (Props):

```
interface ProductCardProps {
  name: string;          // 상품 이름 (필수)
  price: number;         // 가격 (필수)
  inStock: boolean;      // 재고 여부 (필수)
  description?: string;  // 설명 (선택)
}
```

출력 내용:

- `<h2>` 로 상품 이름
- `{price.toLocaleString()}원` 형식으로 가격
- `inStock` true→"재고 있음" / false→"품절"
- `description` 있으면 표시, 없으면 숨김

힌트: `toLocaleString()` 으로 `1,234,567` 형식 변환

점심시간

11:40 – 13:00

오후 첫 세션(13:00)에 돌아오면:

- Session 4: React QuickStart — 틱택토 게임 만들기
- `useState` 훅으로 상태 관리 시작
- 클릭하면 반응하는 인터랙티브 UI 만들기

점심시간 전 체크리스트:

- `npm run dev` 실행 중인 터미널 탭 그대로 유지 (또는 종료 후 오후에 재시작)
- Node.js 설치 안 된 분은 강사에게 말씀해주세요 (점심 중 설치 도움)
- VS Code `Greeting.tsx`, `ProductCard.tsx` 작업 저장 확인

Session 4

React QuickStart: 틱택토 게임 만들기

Props vs State

구분	Props	State
출처	부모 컴포넌트	컴포넌트 자신
변경 주체	부모만 변경 가능	자신이 `setState` 로 변경
재렌더링	부모가 새 Props 내릴 때	`setState` 호출 시
비유	함수의 파라미터	내부 메모리

식당 비유:

Props vs State

구분	Props	State
출처	부모 컴포넌트	컴포넌트 자신
변경 주체	부모만 변경 가능	자신이 `setState` 로 변경
재렌더링	부모가 새 Props 내릴 때	`setState` 호출 시
비유	함수의 파라미터	내부 메모리

식당 비유:

> **Props = 주문서** 손님(부모)이 내용을 결정. 주방(자식)은 받은 대로 처리. 주방이 주문서 내용을 바꿀 수 없음.

Props vs State

구분	Props	State
출처	부모 컴포넌트	컴포넌트 자신
변경 주체	부모만 변경 가능	자신이 `setState` 로 변경
재렌더링	부모가 새 Props 내릴 때	`setState` 호출 시
비유	함수의 파라미터	내부 메모리

식당 비유:

Props = 주문서 손님(부모)이 내용을 결정. 주방(자식)은 받은 대로 처리. 주방이 주문서 내용을 바꿀 수 없음.

State = 재고 목록 주방이 스스로 관리. 재료를 쓸 때마다 주방이 직접 업데이트. 손님이 알 필요 없는 내부 정보.

왜 일반 변수(let)로는 안 되는가

```
// ❌ 이 코드는 동작하지 않습니다
function Counter() {
  let count = 0; // 일반 변수

  function handleClick() {
    count = count + 1;
    console.log('클릭! count =', count); // 콘솔에는 1, 2, 3...으로 증가
  }

  return (
    <div>
      <p>클릭 수: {count}</p>   {/* 화면은 영원히 0에서 바뀌지 않음 */}
      <button onClick={handleClick}>+1</button>
    </div>
```

왜 일반 변수(let)로는 안 되는가

```
  let count = 0; // 일반 변수

  function handleClick() {
    count = count + 1;
    console.log('클릭! count =', count); // 콘솔에는 1, 2, 3...으로 증가
  }

  return (
    <div>
      <p>클릭 수: {count}</p>   {/* 화면은 영원히 0에서 바뀌지 않음 */}
      <button onClick={handleClick}>+1</button>
    </div>
  );
}
```

왜 일반 변수(let)로는 안 되는가

```
  return (
    <div>
      <p>클릭 수: {count}</p>   {/* 화면은 영원히 0에서 바뀌지 않음 */}
      <button onClick={handleClick}>+1</button>
    </div>
  );
}

// ✅ useState를 쓰면 됩니다
function Counter() {
  const [count, setCount] = useState(0);
  // setCount를 호출할 때마다 React가 화면을 다시 그립니다
}
```

왜 일반 변수(let)로는 안 되는가

```
  return (
    <div>
      <p>클릭 수: {count}</p>   {/* 화면은 영원히 0에서 바뀌지 않음 */}
      <button onClick={handleClick}>+1</button>
    </div>
  );
}

// ✅ useState를 쓰면 됩니다
function Counter() {
  const [count, setCount] = useState(0);
  // setCount를 호출할 때마다 React가 화면을 다시 그립니다
}
```

React가 화면을 다시 그리는 조건은 단 두 가지:

1. **State가 변경될 때** (`setState` 함수 호출)
2. **Props가 변경될 때** (부모가 새 값을 내릴 때)

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);       // ← number 추론
const [name, setName]   = useState('');      // ← string 추론
const [flag, setFlag]   = useState(false);   // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);       // ← number 추론
const [name, setName]   = useState('');      // ← string 추론
const [flag, setFlag]   = useState(false);   // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);       // ← number 추론
const [name, setName]   = useState('');      // ← string 추론
const [flag, setFlag]   = useState(false);   // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);       // ← number 추론
const [name, setName]   = useState('');      // ← string 추론
const [flag, setFlag]   = useState(false);   // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

TypeScript 타입 추론 + 제네릭:

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);       // ← number 추론
const [name, setName]   = useState('');      // ← string 추론
const [flag, setFlag]   = useState(false);   // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

`useState` 기본 문법

```
import { useState } from 'react';

function Counter() {
  //      ↓ 현재 값    ↓ 값을 바꾸는 함수     ↓ 초기값
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // 호출하면 React가 화면을 다시 그림
  }
  // ...
}
```

TypeScript 타입 추론 + 제네릭:

```
// 초기값이 있으면 TypeScript가 자동 추론
const [count, setCount] = useState(0);        // ← number 추론
const [name, setName]   = useState('');       // ← string 추론
const [flag, setFlag]   = useState(false);    // ← boolean 추론

// 초기값이 null이거나 타입이 복잡하면 제네릭 명시
const [user, setUser] = useState<User | null>(null);  // User | null
const [items, setItems] = useState<string[]>([]);      // string[]
```

Tic-Tac-Toe: 컴포넌트 트리 설계

왜 틱택토인가? 9개 칸이라는 단순 구조에 React 핵심이 모두 들어있습니다.

```
Board (게임판 전체)
  ├── Square (칸 0)
  ├── Square (칸 1)
  ├── Square (칸 2)
  ├── ...
  └── Square (칸 8)
```

```
// Square: 틱택토 칸 하나
interface SquareProps {
  value: string | null; // 'X', 'O', 또는 null(비어있음)
}
function Square({ value }: SquareProps) {
  return <button className="square">{value}</button>;
}

// Board: 9개 칸을 담는 게임판
function Board() {
  return (
```

Tic-Tac-Toe: 컴포넌트 트리 설계

왜 틱택토인가? 9개 칸이라는 단순 구조에 React 핵심이 모두 들어있습니다.

```
Board (게임판 전체)
  ├── Square (칸 0)
  ├── Square (칸 1)
  ├── Square (칸 2)
  ├── ...
  └── Square (칸 8)
```

Step 1-3: 구조 이해

```
// Square: 틱택토 칸 하나
interface SquareProps {
  value: string | null; // 'X', 'O', 또는 null(비어있음)
}
function Square({ value }: SquareProps) {
  return <button className="square">{value}</button>;
}

// Board: 9개 칸을 담는 게임판
function Board() {
  return (
```

Tic-Tac-Toe: 컴포넌트 트리 설계

왜 틱택토인가? 9개 칸이라는 단순 구조에 React 핵심이 모두 들어있습니다.

```
Board (게임판 전체)
  ├── Square (칸 0)
  ├── Square (칸 1)
  ├── Square (칸 2)
  ├── ...
  └── Square (칸 8)
```

Step 1-3: 구조 이해

```
  return <button className="square">{value}</button>;
}

// Board: 9개 칸을 담는 게임판
function Board() {
  return (
    <div className="board-row">
      <Square value="X" />
      <Square value="O" />
      <Square value={null} />
    </div>
```

Step 4-5: 클릭 이벤트 + Square State

Step 4: onClick 연결

```
function Square({ value }: SquareProps) {
  function handleClick() {
    console.log('Square 클릭됨!');
  }

  return (
    <button className="square" onClick={handleClick}>
      {value}
    </button>
  );
}
```

Step 4-5: 클릭 이벤트 + Square State

Step 4: onClick 연결

```
function Square({ value }: SquareProps) {
  function handleClick() {
    console.log('Square 클릭됨!');
  }

  return (
    <button className="square" onClick={handleClick}>
      {value}
    </button>
  );
}
```

가장 흔한 실수 — 괄호 유무:

```
// ❌ onClick={handleClick()}  ← 렌더링될 때 즉시 실행! 클릭 시가 아님
// ✅ onClick={handleClick}    ← 클릭될 때 실행. 함수 자체를 전달

// ❌ onClick={handleClick(i)} ← 즉시 실행
// ✅ onClick={() => handleClick(i)} ← 클릭될 때 실행. 화살표 함수로 감싸기
```

Step 5 → 문제점 발견: State를 Square에 두면?

Square에 직접 State를 넣어서 클릭 시 X 표시:

```
function Square({ value }: SquareProps) {
  const [clicked, setClicked] = useState(false);

  function handleClick() {
    setClicked(true);
  }

  return (
    <button className="square" onClick={handleClick}>
      {clicked ? 'X' : null}
    </button>
  );
}
```

Step 5 → 문제점 발견: State를 Square에 두면?

Square에 직접 State를 넣어서 클릭 시 X 표시:

```
function Square({ value }: SquareProps) {
  const [clicked, setClicked] = useState(false);

  function handleClick() {
    setClicked(true);
  }

  return (
    <button className="square" onClick={handleClick}>
      {clicked ? 'X' : null}
    </button>
  );
}
```

동작은 하는데… 문제가 있습니다

미니 퀴즈 이 코드로 실제 게임을 완성하려면 어떤 문제가 있을까요? 30초 생각해보세요.

1. 모든 칸이 X만 표시됩니다 → O도 있어야 합니다

Step 5 → 문제점 발견: State를 Square에 두면?

Square에 직접 State를 넣어서 클릭 시 X 표시:

```
function Square({ value }: SquareProps) {
  const [clicked, setClicked] = useState(false);

  function handleClick() {
    setClicked(true);
  }

  return (
    <button className="square" onClick={handleClick}>
      {clicked ? 'X' : null}
    </button>
  );
}
```

동작은 하는데… 문제가 있습니다

미니 퀴즈 이 코드로 실제 게임을 완성하려면 어떤 문제가 있을까요? 30초 생각해보세요.

1. 모든 칸이 X만 표시됩니다 → O도 있어야 합니다

Step 6: State Lifting Up (상태 끌어올리기)

Before → After 다이어그램:

```
Before (각 Square에 State가 흩어져 있음):
  Board
    Square[0] → State: clicked=true  (Board가 모름)
    Square[1] → State: clicked=false (Board가 모름)

After (State를 Board로 끌어올림):
  Board → State: squares = ['X', null, 'O', null, ...] ← Board가 전부 앎
    Square[0] (Props로 value만 받아서 표시)
    Square[1]
```

```
function Board() {
  // 9개 칸의 값 배열: 'X', 'O', 또는 null
  const [squares, setSquares] = useState<(string | null)[]>(
    Array(9).fill(null)   // [null, null, null, ..., null]
  );
  const [xIsNext, setXIsNext] = useState(true); // true=X차례, false=O차례

  function handleClick(i: number) {
    if (squares[i]) return; // 이미 채워진 칸 무시
    const nextSquares = squares.slice(); // ★ 불변성: 복사본 생성
```

Step 6: State Lifting Up (상태 끌어올리기)

Before → After 다이어그램:

```
Before (각 Square에 State가 흩어져 있음):
  Board
    Square[0] → State: clicked=true  (Board가 모름)
    Square[1] → State: clicked=false (Board가 모름)

After (State를 Board로 끌어올림):
  Board → State: squares = ['X', null, 'O', null, ...] ← Board가 전부 앎
    Square[0] (Props로 value만 받아서 표시)
    Square[1]
```

구현: Board에 squares State + handleClick 추가

```
function Board() {
  // 9개 칸의 값 배열: 'X', 'O', 또는 null
  const [squares, setSquares] = useState<(string | null)[]>(
    Array(9).fill(null)   // [null, null, null, ..., null]
  );
  const [xIsNext, setXIsNext] = useState(true); // true=X차례, false=O차례

  function handleClick(i: number) {
    if (squares[i]) return; // 이미 채워진 칸 무시
    const nextSquares = squares.slice(); // ★ 불변성: 복사본 생성
```

Step 6: State Lifting Up (상태 끌어올리기)

Before → After 다이어그램:

```
Before (각 Square에 State가 흩어져 있음):
  Board
    Square[0] → State: clicked=true  (Board가 모름)
    Square[1] → State: clicked=false (Board가 모름)

After (State를 Board로 끌어올림):
  Board → State: squares = ['X', null, 'O', null, ...] ← Board가 전부 앎
    Square[0] (Props로 value만 받아서 표시)
    Square[1]
```

구현: Board에 squares State + handleClick 추가

```
    Array(9).fill(null)   // [null, null, null, ..., null]
  );
  const [xIsNext, setXIsNext] = useState(true); // true=X차례, false=O차례

  function handleClick(i: number) {
    if (squares[i]) return; // 이미 채워진 칸 무시
    const nextSquares = squares.slice(); // ★ 불변성: 복사본 생성
```

Step 6 완성: Square에 Callback Props 전달

Square의 Props 수정 — 클릭 핸들러도 Props로 받기:

```
interface SquareProps {
  value: string | null;
  onSquareClick: () => void; // 파라미터 없고 반환값 없는 함수 타입
}

function Square({ value, onSquareClick }: SquareProps) {
  return (
    <button className="square" onClick={onSquareClick}>
      {value}
    </button>
  );
}

// Board의 return 안 — 각 Square에 값과 핸들러 전달
<div className="board-row">
  <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
  <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
  <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
</div>
```

Step 6 완성: Square에 Callback Props 전달

Square의 Props 수정 — 클릭 핸들러도 Props로 받기:

```
interface SquareProps {
  value: string | null;
  onSquareClick: () => void; // 파라미터 없고 반환값 없는 함수 타입
}

function Square({ value, onSquareClick }: SquareProps) {
  return (
    <button className="square" onClick={onSquareClick}>
      {value}
    </button>
  );
}

// Board의 return 안 — 각 Square에 값과 핸들러 전달
<div className="board-row">
  <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
  <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
  <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
</div>
```

Step 6 완성: Square에 Callback Props 전달

Square의 Props 수정 — 클릭 핸들러도 Props로 받기:

```
interface SquareProps {
  value: string | null;
  onSquareClick: () => void; // 파라미터 없고 반환값 없는 함수 타입
}

function Square({ value, onSquareClick }: SquareProps) {
  return (
    <button className="square" onClick={onSquareClick}>
      {value}
    </button>
  );
}

// Board의 return 안 — 각 Square에 값과 핸들러 전달
<div className="board-row">
  <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
  <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
  <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
</div>
```

`() => handleClick(0)` 왜 이렇게 쓰나?

불변성(Immutability)이 중요한 이유

```
// ❌ 잘못된 방법 — 기존 배열을 직접 수정 (Mutation)
function handleClick(i: number) {
  squares[i] = 'X';       // 배열 직접 수정 → 같은 메모리 주소
  setSquares(squares);    // React: "이전이랑 같은 배열? 변경 없음" → 재렌더링 없음!
}

// ✅ 올바른 방법 — 새 배열 생성 (Immutability)
function handleClick(i: number) {
  const nextSquares = squares.slice(); // 새 배열 생성 → 다른 메모리 주소
  nextSquares[i] = 'X';
  setSquares(nextSquares); // React: "새 배열이네! 변경됨" → 재렌더링
}
```

불변성(Immutability)이 중요한 이유

```
// ❌ 잘못된 방법 — 기존 배열을 직접 수정 (Mutation)
function handleClick(i: number) {
  squares[i] = 'X';       // 배열 직접 수정 → 같은 메모리 주소
  setSquares(squares);    // React: "이전이랑 같은 배열? 변경 없음" → 재렌더링 없음!
}

// ✅ 올바른 방법 — 새 배열 생성 (Immutability)
function handleClick(i: number) {
  const nextSquares = squares.slice(); // 새 배열 생성 → 다른 메모리 주소
  nextSquares[i] = 'X';
  setSquares(nextSquares); // React: "새 배열이네! 변경됨" → 재렌더링
}
```

불변성(Immutability)이 중요한 이유

```
// ❌ 잘못된 방법 — 기존 배열을 직접 수정 (Mutation)
function handleClick(i: number) {
  squares[i] = 'X';       // 배열 직접 수정 → 같은 메모리 주소
  setSquares(squares);    // React: "이전이랑 같은 배열? 변경 없음" → 재렌더링 없음!
}

// ✅ 올바른 방법 — 새 배열 생성 (Immutability)
function handleClick(i: number) {
  const nextSquares = squares.slice(); // 새 배열 생성 → 다른 메모리 주소
  nextSquares[i] = 'X';
  setSquares(nextSquares); // React: "새 배열이네! 변경됨" → 재렌더링
}
```

React 배열 불변성 치트시트:

```
const arr = [1, 2, 3, 4, 5];

// 추가 — spread 연산자
const added   = [ ...arr, 6];                        // [1,2,3,4,5,6]
// 삭제 — filter (새 배열 반환)
const removed = arr.filter(n => n !== 3);            // [1,2,4,5]
// 수정 — map (새 배열 반환)
const updated = arr.map(n => n === 3 ? 99 : n);     // [1,2,99,4,5]
```

Thinking in React

React로 생각하는 5단계

Thinking in React 5단계

React로 UI를 만들 때 막막하다면? 이 5단계를 따르세요.

단계	핵심 질문	우리 예제에서
Step 1: 컴포넌트 분해	UI를 어떻게 나눌 것인가?	5개 컴포넌트로 분해
Step 2: 정적 버전	Props만으로 UI를 만들 수 있는가?	State 없이 UI 먼저 완성
Step 3: 최소 State	어떤 데이터가 진짜 State인가?	검색어, 체크 여부만 State
Step 4: State 위치	어느 컴포넌트에 State를 두는가?	공통 부모에 배치
Step 5: 역방향 흐름	자식이 부모 State를 어떻게 바꾸는가?	Callback Props 전달

Thinking in React 5단계

React로 UI를 만들 때 막막하다면? 이 5단계를 따르세요.

단계	핵심 질문	우리 예제에서
Step 1: 컴포넌트 분해	UI를 어떻게 나눌 것인가?	5개 컴포넌트로 분해
Step 2: 정적 버전	Props만으로 UI를 만들 수 있는가?	State 없이 UI 먼저 완성
Step 3: 최소 State	어떤 데이터가 진짜 State인가?	검색어, 체크 여부만 State
Step 4: State 위치	어느 컴포넌트에 State를 두는가?	공통 부모에 배치
Step 5: 역방향 흐름	자식이 부모 State를 어떻게 바꾸는가?	Callback Props 전달

오늘 만들 UI: 상품 검색 필터

```
검색: [___________________________]
☑ 재고 있는 상품만 보기
```

Step 1 : UI를 컴포넌트로 분해

단일 책임 원칙: 하나의 컴포넌트는 하나의 일만 해야 합니다.

```
┌── FilterableProductTable ──────────────────────────────┐
│                                                        │
│  ┌── SearchBar ──────────────────────────────────┐     │
│  │ 검색: [_________________]                     │     │
│  │ ☑ 재고 있는 상품만 보기                       │     │
│  └───────────────────────────────────────────────┘     │
│                                                        │
│  ┌── ProductTable ───────────────────────────────┐     │
│  │ 이름            가격                          │     │
│  │  ┌── ProductCategoryRow ──────────────┐       │     │
│  │  │ 과일                               │       │     │
│  │  └────────────────────────────────────┘       │     │
│  │  ┌── ProductRow ──────────────────────┐       │     │
│  │  │ 사과            ₩1,500             │       │     │
│  │  └────────────────────────────────────┘       │     │
│  └───────────────────────────────────────────────┘     │
└────────────────────────────────────────────────────────┘
```

Step 1 : UI를 컴포넌트로 분해

단일 책임 원칙: 하나의 컴포넌트는 하나의 일만 해야 합니다.

```
┌── FilterableProductTable ──────────────────────────────┐
│                                                        │
│  ┌── SearchBar ──────────────────────────────┐         │
│  │ 검색: [__________________]                │         │
│  │ ☑ 재고 있는 상품만 보기                   │         │
│  └───────────────────────────────────────────┘         │
│                                                        │
│  ┌── ProductTable ───────────────────────────┐         │
│  │ 이름            가격                      │         │
│  │  ┌── ProductCategoryRow ──────────────┐   │         │
│  │  │ 과일                               │   │         │
│  │  └────────────────────────────────────┘   │         │
│  │  ┌── ProductRow ──────────────────────┐   │         │
│  │  │ 사과            ₩1,500             │   │         │
│  │  └────────────────────────────────────┘   │         │
│  └───────────────────────────────────────────┘         │
└────────────────────────────────────────────────────────┘
```

컴포넌트 계층 구조:

```
FilterableProductTable   ← 전체를 감싸는 최상위
  ├── SearchBar          ← 검색창 + 체크박스
```

Step 2: 정적 버전 만들기

State 없이 Props만으로 먼저 UI를 완성합니다.

왜? State와 동작을 동시에 고민하면 복잡해집니다. 먼저 "어떻게 보여야 하는가"를 해결하세요.

```typescript
// 타입 정의 먼저
interface Product {
  category: string; // '과일', '채소'
  price: string;    // 'W1,500', '품절'
  stocked: boolean; // 재고 여부
  name: string;     // 상품 이름
}

// 정적 데이터
const PRODUCTS: Product[] = [
  { category: '과일', price: 'W1,500', stocked: true,  name: '사과'   },
  { category: '과일', price: '품절',   stocked: false, name: '용과'   },
  { category: '채소', price: 'W2,000', stocked: true,  name: '시금치' },
  { category: '채소', price: 'W3,000', stocked: true,  name: '호박'   },
];
```

Step 2: 정적 버전 만들기

State 없이 Props만으로 먼저 UI를 완성합니다.

왜? State와 동작을 동시에 고민하면 복잡해집니다. 먼저 "어떻게 보여야 하는가"를 해결하세요.

```
// 타입 정의 먼저
interface Product {
  category: string; // '과일', '채소'
  price: string;    // 'W1,500', '품절'
  stocked: boolean; // 재고 여부
  name: string;     // 상품 이름
}

// 정적 데이터
const PRODUCTS: Product[] = [
  { category: '과일', price: 'W1,500', stocked: true,  name: '사과'   },
  { category: '과일', price: '품절',   stocked: false, name: '용과'   },
  { category: '채소', price: 'W2,000', stocked: true,  name: '시금치' },
  { category: '채소', price: 'W3,000', stocked: true,  name: '호박'   },
];
```

Step 2: 정적 버전 만들기

State 없이 Props만으로 먼저 UI를 완성합니다.

왜? State와 동작을 동시에 고민하면 복잡해집니다. 먼저 "어떻게 보여야 하는가"를 해결하세요.

```
// 타입 정의 먼저
interface Product {
  category: string; // '과일', '채소'
  price: string;    // 'W1,500', '품절'
  stocked: boolean; // 재고 여부
  name: string;     // 상품 이름
}

// 정적 데이터
const PRODUCTS: Product[] = [
  { category: '과일', price: 'W1,500', stocked: true,  name: '사과'   },
  { category: '과일', price: '품절',   stocked: false, name: '용과'   },
  { category: '채소', price: 'W2,000', stocked: true,  name: '시금치' },
  { category: '채소', price: 'W3,000', stocked: true,  name: '호박'   },
];
```

ProductRow: 재고 없는 상품 빨간색

Step 3: 최소 State 식별

State는 최소한으로 유지합니다. DRY 원칙.

State가 아닌 것 판별 기준:

Step 3: 최소 State 식별

State는 최소한으로 유지합니다. DRY 원칙.

State가 아닌 것 판별 기준:

1. 시간이 지나도 변하지 않는가? → **State 불필요** (상수)
2. 부모에게서 Props로 받는가? → **State 불필요** (Props 그대로 사용)
3. 다른 State나 Props로 계산할 수 있는가? → **State 불필요** (파생값)

Step 3: 최소 State 식별

State는 최소한으로 유지합니다. DRY 원칙.

State가 아닌 것 판별 기준:

1. 시간이 지나도 변하지 않는가? → **State 불필요** (상수)
2. 부모에게서 Props로 받는가? → **State 불필요** (Props 그대로 사용)
3. 다른 State나 Props로 계산할 수 있는가? → **State 불필요** (파생값)

우리 예제 분석:

데이터	State?	이유
상품 목록 원본	❌	외부 고정 데이터 (상수 또는 Props)
검색어 (`filterText`)	✅	사용자 입력으로 변하고 UI에 반영
체크 여부 (`inStockOnly`)	✅	사용자 행동으로 변하고 UI에 반영

Step 3: 최소 State 식별

State는 최소한으로 유지합니다. DRY 원칙.

State가 아닌 것 판별 기준:

1. 시간이 지나도 변하지 않는가? → **State 불필요** (상수)
2. 부모에게서 Props로 받는가? → **State 불필요** (Props 그대로 사용)
3. 다른 State나 Props로 계산할 수 있는가? → **State 불필요** (파생값)

우리 예제 분석:

데이터	State?	이유
상품 목록 원본	❌	외부 고정 데이터 (상수 또는 Props)
검색어 (`filterText`)	✅	사용자 입력으로 변하고 UI에 반영
체크 여부 (`inStockOnly`)	✅	사용자 행동으로 변하고 UI에 반영

Step 4: State 위치 결정

규칙: State는 그것을 필요로 하는 **모든 컴포넌트의 공통 부모**에 위치해야 합니다.

`filterText` 와 `inStockOnly` 를 누가 필요로 하는가?

Step 4: State 위치 결정

규칙: State는 그것을 필요로 하는 **모든 컴포넌트의 공통 부모**에 위치해야 합니다.

`filterText` 와 `inStockOnly` 를 누가 필요로 하는가?

```
FilterableProductTable  ← SearchBar와 ProductTable의 공통 부모 ✅
  ├── SearchBar          (입력값 표시에 filterText, inStockOnly 필요)
  └── ProductTable       (필터링에 filterText, inStockOnly 필요)
```

→ **FilterableProductTable에 State 배치!**

Step 4: State 위치 결정

규칙: State는 그것을 필요로 하는 **모든 컴포넌트의 공통 부모**에 위치해야 합니다.

`filterText` 와 `inStockOnly` 를 누가 필요로 하는가?

```
FilterableProductTable  ← SearchBar와 ProductTable의 공통 부모 ✅
  ├── SearchBar          (입력값 표시에 filterText, inStockOnly 필요)
  └── ProductTable       (필터링에 filterText, inStockOnly 필요)
```

→ FilterableProductTable에 State 배치!

```
function FilterableProductTable() {
  // Step 3에서 결정한 딱 두 개의 State
  const [filterText, setFilterText]   = useState('');
  const [inStockOnly, setInStockOnly] = useState(false);

  return (
    <div>
      <SearchBar
        filterText={filterText}
        inStockOnly={inStockOnly}
        // Step 5: 변경 함수도 Props로 전달 (역방향 흐름)
        onFilterTextChange={setFilterText}
        onInStockOnlyChange={setInStockOnly}
```

Step 5: 역방향 데이터 흐름

React 데이터는 항상 **부모 → 자식** (단방향). 자식이 부모 State를 직접 수정할 수 없습니다.

대신 부모가 **State 변경 함수를 자식에게 Props로 전달**합니다.

```
interface SearchBarProps {
  filterText: string;
  inStockOnly: boolean;
  onFilterTextChange: (value: string) => void;
  onInStockOnlyChange: (value: boolean) => void;
}

function SearchBar({ filterText, inStockOnly, onFilterTextChange, onInStockOnlyChange }: SearchBarProps) {
  return (
    <form>
      <input
        type="text"
        value={filterText}                              // State가 현재 값 결정
        onChange={(e) => onFilterTextChange(e.target.value)} // 타이핑 시 부모 State 변경
      />
      <input
        type="checkbox"
        checked={inStockOnly}                           // checked 사용! (value 아님)
        onChange={(e) => onInStockOnlyChange(e.target.checked)}
```

Step 5: 역방향 데이터 흐름

React 데이터는 항상 **부모 → 자식** (단방향). 자식이 부모 State를 직접 수정할 수 없습니다.

대신 부모가 **State 변경 함수를 자식에게 Props로 전달**합니다.

SearchBar — Callback Props 구현:

```
interface SearchBarProps {
  filterText: string;
  inStockOnly: boolean;
  onFilterTextChange: (value: string) => void;
  onInStockOnlyChange: (value: boolean) => void;
}

function SearchBar({ filterText, inStockOnly, onFilterTextChange, onInStockOnlyChange }: SearchBarProps) {
  return (
    <form>
      <input
        type="text"
        value={filterText}                              // State가 현재 값 결정
        onChange={(e) => onFilterTextChange(e.target.value)} // 타이핑 시 부모 State 변경
      />
      <input
        type="checkbox"
        checked={inStockOnly}                           // checked 사용! (value 아님)
        onChange={(e) => onInStockOnlyChange(e.target.checked)}
```

Step 5: 역방향 데이터 흐름

React 데이터는 항상 **부모 → 자식** (단방향). 자식이 부모 State를 직접 수정할 수 없습니다.

대신 부모가 **State 변경 함수를 자식에게 Props로 전달**합니다.

SearchBar — Callback Props 구현:

```
  onInStockOnlyChange: (value: boolean) => void;
}

function SearchBar({ filterText, inStockOnly, onFilterTextChange, onInStockOnlyChange }: SearchBarProps) {
  return (
    <form>
      <input
        type="text"
        value={filterText}                              // State가 현재 값 결정
        onChange={(e) => onFilterTextChange(e.target.value)} // 타이핑 시 부모 State 변경
      />
      <input
        type="checkbox"
        checked={inStockOnly}                           // checked 사용! (value 아님)
        onChange={(e) => onInStockOnlyChange(e.target.checked)}
      />
    </form>
  );
}
```

제어 컴포넌트 패턴 + 완성

Controlled Component: `value` 와 `onChange` 를 반드시 쌍으로 사용합니다.

```
// ✅ 완전한 제어 컴포넌트 — 양방향 연결
<input
  type="text"
  value={filterText}                              // State → input (표시)
  onChange={(e) => setFilterText(e.target.value)} // input → State (변경)
/>
```

제어 컴포넌트 패턴 + 완성

Controlled Component: `value` 와 `onChange` 를 반드시 쌍으로 사용합니다.

```
// ✅ 완전한 제어 컴포넌트 — 양방향 연결
<input
  type="text"
  value={filterText}                              // State → input (표시)
  onChange={(e) => setFilterText(e.target.value)} // input → State (변경)
/>
```

흔한 실수:

- `onChange` 만 있고 `value` 없음 → 비제어 컴포넌트. 프로그래밍 방식으로 값 초기화 불가
- `value` 만 있고 `onChange` 없음 → 타이핑해도 값이 안 바뀜. React 경고 발생
  - `"Warning: You provided a 'value' prop without an 'onChange' handler."`

제어 컴포넌트 패턴 + 완성

Controlled Component: `value` 와 `onChange` 를 반드시 쌍으로 사용합니다.

```
// ✅ 완전한 제어 컴포넌트 — 양방향 연결
<input
  type="text"
  value={filterText}                              // State → input (표시)
  onChange={(e) => setFilterText(e.target.value)} // input → State (변경)
/>
```

흔한 실수:

- `onChange` 만 있고 `value` 없음 → 비제어 컴포넌트. 프로그래밍 방식으로 값 초기화 불가
- `value` 만 있고 `onChange` 없음 → 타이핑해도 값이 안 바뀜. React 경고 발생
  - `"Warning: You provided a 'value' prop without an 'onChange' handler."`

완성 후 동작 확인:

1. "사과" 입력 → 사과만 표시

Session 4 마무리 + Social Dashboard 연결

오늘 배운 핵심 패턴 → Session 5에서 그대로 사용됩니다!

오늘 패턴	Social Dashboard 적용
`useState` + `[]` 초기값	게시글 목록, 사용자 목록 초기화
`onClick` 이벤트 핸들러	게시글 삭제, Todo 토글 버튼
`filter` 불변성 패턴	`posts.filter(p => p.id !== id)` 삭제
`map` 불변성 패턴	`todos.map(t => t.id === id ? {...t, done: !t.done} : t)`
`[...arr, 새항목]` spread	새 게시글 추가
State Lifting Up	여러 섹션에서 공유하는 선택된 userId
Callback Props	PostList → App으로 삭제 요청 전달

Session 4 마무리 + Social Dashboard 연결

오늘 배운 핵심 패턴 → Session 5에서 그대로 사용됩니다!

오늘 패턴	Social Dashboard 적용
`useState` + `[]` 초기값	게시글 목록, 사용자 목록 초기화
`onClick` 이벤트 핸들러	게시글 삭제, Todo 토글 버튼
`filter` 불변성 패턴	`posts.filter(p => p.id !== id)` 삭제
`map` 불변성 패턴	`todos.map(t => t.id === id ? {...t, done: !t.done} : t)`
`[...arr, 새항목]` spread	새 게시글 추가
State Lifting Up	여러 섹션에서 공유하는 선택된 userId
Callback Props	PostList → App으로 삭제 요청 전달

Session 5: 종합 실습

Social Dashboard

2시간 실습 (15:00 – 17:00)
JSONPlaceholder REST API 연동 · TypeScript + React 통합 실습

앱 소개 & 목표

완성하면 이런 앱이 됩니다

왼쪽 사이드바 — 사용자 10명 카드 목록
클릭하면 오른쪽에 해당 사용자 정보 표시

오른쪽 메인 영역 — Posts / Todos / Albums 탭
선택된 사용자의 게시글, 할 일, 앨범 표시

상단 통계 카드 — Users, Posts, Comments 등
(도전 과제 8 완성 시 활성화)

필수 과제 6개

- 1. 사용자 목록 (강사 시연)
- 2. 게시글 목록
- 3. 게시글 추가 (POST)
- 4. 게시글 삭제 (DELETE)
- 5. Todo 완료 토글 (PATCH)
- 6. 댓글 표시

도전 과제 5개

Promise.all · useMemo · AlbumGallery · Skeleton · StatsUpdateFn

핵심 개념 리뷰: useState + TypeScript 제네릭

> 오늘 실습에서 가장 많이 쓰는 두 패턴을 빠르게 복습합니다

```
// 세션 1에서 본 제네릭 — T라는 자리표시자 타입
function identity<T>(value: T): T { return value; }

// useState도 동일한 제네릭 패턴
// useState<T>(): T 타입의 값을 상태로 관리
const [users, setUsers] = useState<User[]>([]);
//                                 ^^^^^^^^
//              T = User[]  →  초기값 []는 User[] 타입

// null이 될 수 있는 상태: Union 타입
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//          "User 객체이거나, null이거나" — 처음엔 아무도 선택 안 됨
```

핵심 개념 리뷰: useState + TypeScript 제네릭

> 오늘 실습에서 가장 많이 쓰는 두 패턴을 빠르게 복습합니다

```
// 세션 1에서 본 제네릭 — T라는 자리표시자 타입
function identity<T>(value: T): T { return value; }

// useState도 동일한 제네릭 패턴
// useState<T>(): T 타입의 값을 상태로 관리
const [users, setUsers] = useState<User[]>([]);
//                                 ^^^^^^^^
//              T = User[]  →  초기값 []는 User[] 타입

// null이 될 수 있는 상태: Union 타입
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//          "User 객체이거나, null이거나" — 처음엔 아무도 선택 안 됨
```

핵심 개념 리뷰: useState + TypeScript 제네릭

> 오늘 실습에서 가장 많이 쓰는 두 패턴을 빠르게 복습합니다

```
// 세션 1에서 본 제네릭 — T라는 자리표시자 타입
function identity<T>(value: T): T { return value; }

// useState도 동일한 제네릭 패턴
// useState<T>(): T 타입의 값을 상태로 관리
const [users, setUsers] = useState<User[]>([]);
//                                 ^^^^^^^^
//              T = User[]  →  초기값 []는 User[] 타입

// null이 될 수 있는 상태: Union 타입
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//          "User 객체이거나, null이거나" — 처음엔 아무도 선택 안 됨
```

핵심 개념 리뷰: useState + TypeScript 제네릭

오늘 실습에서 가장 많이 쓰는 두 패턴을 빠르게 복습합니다

```
// 세션 1에서 본 제네릭 — T라는 자리표시자 타입
function identity<T>(value: T): T { return value; }

// useState도 동일한 제네릭 패턴
// useState<T>(): T 타입의 값을 상태로 관리
const [users, setUsers] = useState<User[]>([]);
//                                 ^^^^^^^^
//              T = User[]  →  초기값 []는 User[] 타입

// null이 될 수 있는 상태: Union 타입
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//          "User 객체이거나, null이거나" — 처음엔 아무도 선택 안 됨
```

왜 제네릭을 반드시 써야 하나요?

`useState([])` → TypeScript가 `never[]` 로 추론 → 아무것도 넣을 수 없는 배열이 됨!

`useState<User[]>([])` → "이 배열엔 User 객체만 들어간다" 명시 → 자동완성 + 타입 검사 활성화

핵심 개념 리뷰: useEffect 의존성 배열

의존성 배열 3가지 형태

```
// 형태 1: 빈 배열 — 마운트 시 딱 한 번만
useEffect(() => {
  fetchUsers();
}, []);

// 형태 2: 값 배열 — selectedUser 바뀔 때마다
useEffect(() => {
  if (!selectedUser) return;
  fetchPosts();
}, [selectedUser]);

// 형태 3: 배열 없음 — 매 렌더링마다 (거의 사용 안 함)
useEffect(() => { console.log("매번!"); });
```

useEffect 안에서 async 사용 — 잘못된 방법

```
// ❌ 이렇게 하지 마세요
useEffect(async () => {
  const data = await fetch(url);
}, []);
```

올바른 방법 — 내부에 async 함수 정의

```
// ✅ 이 패턴을 외워두세요
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch(url);
    const data = await res.json();
    setState(data);
  };
  fetchData(); // ← 반드시 호출!
}, []);
```

핵심 개념 리뷰: useEffect 의존성 배열

의존성 배열 3가지 형태

```
// 형태 1: 빈 배열 — 마운트 시 딱 한 번만
useEffect(() => {
  fetchUsers();
}, []);

// 형태 2: 값 배열 — selectedUser 바뀔 때마다
useEffect(() => {
  if (!selectedUser) return;
  fetchPosts();
}, [selectedUser]);

// 형태 3: 배열 없음 — 매 렌더링마다 (거의 사용 안 함)
useEffect(() => { console.log("매번!"); });
```

useEffect 안에서 async 사용 — 잘못된 방법

```
// ❌ 이렇게 하지 마세요
useEffect(async () => {
  const data = await fetch(url);
}, []);
```

올바른 방법 — 내부에 async 함수 정의

```
// ✅ 이 패턴을 외워두세요
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch(url);
    const data = await res.json();
    setState(data);
  };
  fetchData(); // ← 반드시 호출!
}, []);
```

핵심 개념 리뷰: useEffect 의존성 배열

의존성 배열 3가지 형태

```
// 형태 1: 빈 배열 — 마운트 시 딱 한 번만
useEffect(() => {
  fetchUsers();
}, []);

// 형태 2: 값 배열 — selectedUser 바뀔 때마다
useEffect(() => {
  if (!selectedUser) return;
  fetchPosts();
}, [selectedUser]);

// 형태 3: 배열 없음 — 매 렌더링마다 (거의 사용 안 함)
useEffect(() => { console.log("매번!"); });
```

useEffect 안에서 async 사용 — 잘못된 방법

```
// ❌ 이렇게 하지 마세요
useEffect(async () => {
  const data = await fetch(url);
}, []);
```

올바른 방법 — 내부에 async 함수 정의

```
// ✅ 이 패턴을 외워두세요
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch(url);
    const data = await res.json();
    setState(data);
  };
  fetchData(); // ← 반드시 호출!
}, []);
```

프로젝트 구조 & 셋업

```
social-dashboard-starter/
├── src/
│   ├── App.tsx          ← 메인 파일 (주로 여기서 작업)
│   ├── types.ts         ← User, Post, Todo 등 타입 정의
│   ├── components/
│   │   ├── UserCard.tsx
│   │   ├── PostCard.tsx
│   │   ├── PostForm.tsx
│   │   ├── TodoSection.tsx  ← Task 5에서 수정
│   │   ├── CommentList.tsx  ← Task 6에서 수정
│   │   └── AlbumGallery.tsx ← 도전 7
│   └── main.tsx
└── solutions/           ← 정답 코드 (막히면 참고)
```

터미널에서 실행

```
cd social-dashboard-starter
npm install
npm run dev
# → http://localhost:5173 에서 확인
```

셋업 체크리스트

- ☐ 브라우저에 스켈레톤 로딩 화면이 보인다
- ☐ VS Code에서 src/App.tsx를 열었다
- ☐ TODO 주석이 보인다
- ☐ src/types.ts를 열어 인터페이스를 확인했다

I DO — Task 1 : 사용자 목록 표시

강사 시연 (약 30분)

지금부터 강사가 직접 코딩합니다. 여러분은 타이핑하지 말고 눈으로 먼저 확인하세요.
코드를 한 줄씩 입력할 테니, 각 줄이 무엇을 하는지 집중해서 보십시오.

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

```
// State 선언 영역 (파일 상단)
const [users, setUsers] = useState<User[]>([]);
//      ^^^^^  ^^^^^^^^              ^^^^^^  ^^
//  현재 값   변경 함수         타입: User배열  초기값
```

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

```
// State 선언 영역 (파일 상단)
const [users, setUsers] = useState<User[]>([]);
//      ^^^^^  ^^^^^^^^              ^^^^^^  ^^
//  현재 값   변경 함수         타입: User배열  초기값
```

각 부분의 의미

- `const [users, setUsers]` — 배열 구조분해. useState는 항상 [값, 변경함수] 쌍을 반환
- `useState<User[]>` — 제네릭. "이 상태는 User 객체의 배열"임을 TypeScript에게 명시
- `([])` — 초기값. 처음엔 빈 배열, API 응답 받으면 채워짐

Step 1-a: users 상태 선언

`src/App.tsx` 를 열고 `// TODO (필수 1-a)` 주석을 찾습니다

```
// State 선언 영역 (파일 상단)
const [users, setUsers] = useState<User[]>([]);
//      ^^^^^  ^^^^^^^^              ^^^^^^  ^^
//  현재 값   변경 함수         타입: User배열  초기값
```

각 부분의 의미

- `const [users, setUsers]` — 배열 구조분해. useState는 항상 [값, 변경함수] 쌍을 반환
- `useState<User[]>` — 제네릭. "이 상태는 User 객체의 배열"임을 TypeScript에게 명시
- `([])` — 초기값. 처음엔 빈 배열, API 응답 받으면 채워짐

에디터에서 확인: `users.` 입력 후 자동완성 → `.length`, `.map`, `.filter` 등 표시
`users[0].` 입력 후 자동완성 → `.name`, `.email`, `.id` 등 User interface 속성 표시

Step 1-b: selectedUser 상태 선언

바로 아래 줄에 이어서 입력합니다

Step 1-b: selectedUser 상태 선언

바로 아래 줄에 이어서 입력합니다

Step 1-b: selectedUser 상태 선언

바로 아래 줄에 이어서 입력합니다

```
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//                                      "User이거나 null이거나" — Union 타입
```

Step 1-b: selectedUser 상태 선언

바로 아래 줄에 이어서 입력합니다

```
const [selectedUser, setSelectedUser] = useState<User | null>(null);
//                                                ^^^^^^^^^^^
//                                  "User이거나 null이거나" — Union 타입
```

Union 타입 복습

세션 1에서 배운 `string | number` 패턴과 동일

지금은 "User 객체 *또는* null"

초기값 null → 처음엔 아무도 선택 안 됨

카드 클릭 → setSelectedUser(user) → User 객체가 됨

임시 변수 반드시 삭제!

파일 아래쪽에 이런 줄이 있습니다:

```
// 이 두 줄을 찾아서 삭제하세요
const users = JSON.parse("[]") as User[];
const selectedUser = JSON.parse("null") as User | nul
```

useState와 변수 이름이 겹쳐 TypeScript 오류 발생!

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

Step 1-c: useEffect + fetch 구현 (1/2)

`// TODO (필수 1-c)` 주석 아래의 빈 useEffect를 채웁니다

```
useEffect(() => {
  const fetchUsers = async () => {          // ① 내부에 async 함수 정의
    const res = await fetch(`${API}/users`); // ② HTTP GET 요청
    const data: User[] = await res.json();   // ③ JSON 파싱 + 타입 명시
    setUsers(data);                          // ④ 상태 업데이트
    setLoading(false);                       // ⑤ 로딩 종료
  };
  fetchUsers();  // ⑥ 반드시 호출 (없으면 함수 정의만 되고 실행 안 됨)
}, []);          // ⑦ 빈 배열 = 마운트 시 한 번만
```

API 상수가 뭔가요?

파일 맨 위에 `const API = 'https://jsonplaceholder.typicode.com'` 이 선언되어 있습니다.

백틱 템플릿 리터럴로 조합하면 → `https://jsonplaceholder.typicode.com/users`

Step 1-c: 저장 후 브라우저 확인

저장(Ctrl+S / Cmd+S) 후 브라우저로 이동합니다

브라우저에서 확인할 것

Step 1-c: 저장 후 브라우저 확인

저장(Ctrl+S / Cmd+S) 후 브라우저로 이동합니다

브라우저에서 확인할 것

왼쪽 사이드바에 잠깐 스켈레톤이 보이다가
사용자 카드 10개가 나타난다

개발자 도구(F12) → Network 탭 →
`users` 요청이 Status 200 OK

Step 1-c: 저장 후 브라우저 확인

저장(Ctrl+S / Cmd+S) 후 브라우저로 이동합니다

브라우저에서 확인할 것

왼쪽 사이드바에 잠깐 스켈레톤이 보이다가
사용자 카드 10개가 나타난다

개발자 도구(F12) → Network 탭 →
`users` 요청이 Status 200 OK

이 시점에서 흔한 실수

- fetchUsers() 호출 줄 누락 → 계속 로딩만 됨
- setLoading(false) 누락 → 데이터는 있지만 스켈레톤 계속 표시
- 의존성 배열 [] 누락 → 무한 루프 (계속 fetch 반복)

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

Step 1-d: UserCard 렌더링

렌더링 부분으로 스크롤 → `TODO (필수 1-d)` 주석 아래 찾기

변경 전 (삭제할 것)

```
{/* TODO (필수 1-d): ... */}
<p className="text-sm text-slate-400">
  TODO: UserCard를 렌더링하세요
</p>
```

변경 후 (입력할 것)

```
{users.map((user) => (
  <UserCard
    key={user.id}
    user={user}
    selected={selectedUser?.id === user.id}
    onClick={setSelectedUser}
  />
))}
```

각 prop 설명

- `key={user.id}` — React 리스트 고유 식별자. 1~10 정수. 없으면 경고 발생
- `selected={selectedUser?.id === user.id}` — 옵셔널 체이닝. null이면 false, 같은 id면 true → 강조 표시
- `onClick={setSelectedUser}` — 콜백 props 패턴. 카드 클릭 시 App의 state를 변경

Task 1 완료 확인 체크리스트

브라우저에서 확인

코드 체크

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시

코드 체크

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시

코드 체크

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시
- ✅ 선택된 카드가 파란 테두리로 강조

코드 체크

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시
- ✅ 선택된 카드가 파란 테두리로 강조
- ✅ 다른 카드 클릭 시 강조가 이동

코드 체크

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시
- ✅ 선택된 카드가 파란 테두리로 강조
- ✅ 다른 카드 클릭 시 강조가 이동

코드 체크

- □ 임시 변수 두 줄 (JSON.parse) 삭제 완료

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시
- ✅ 선택된 카드가 파란 테두리로 강조
- ✅ 다른 카드 클릭 시 강조가 이동

코드 체크

- □ 임시 변수 두 줄 (JSON.parse) 삭제 완료
- □ fetchUsers() 호출 줄 있음

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

Task 1 완료 확인 체크리스트

브라우저에서 확인

- ✅ 왼쪽 사이드바에 사용자 10명 카드 표시
- ✅ 카드 클릭 시 오른쪽에 이름·이메일·회사 표시
- ✅ 선택된 카드가 파란 테두리로 강조
- ✅ 다른 카드 클릭 시 강조가 이동

코드 체크

- □ 임시 변수 두 줄 (JSON.parse) 삭제 완료
- □ fetchUsers() 호출 줄 있음
- □ 의존성 배열 `[]` 있음

아직 Posts 탭이 비어있는 것이 정상입니다.
Task 2에서 게시글 fetch를 구현합니다.

YOU DO — Task 2: 게시글 목록

난이도: ★★☆☆☆ | 예상 시간: 15분
Task 1-c 패턴을 거의 그대로 응용

구현할 TODO

- `(필수 2-a)` : posts 상태 선언 — `Post[]` 타입
- `(필수 2-b)` : selectedUser 의존 useEffect
- 렌더링 부분: PostCard 주석 해제

핵심 힌트

```
useEffect(() => {
  if (!selectedUser) return;  // null이면 종료
  const fetchPosts = async () => {
    // URL: `${API}/posts?userId=${selectedUser.id}`
    // 타입: Post[]
    // setter: setPosts
  };
```

Task 1과의 차이점 2가지

1. **의존성 배열**: `[]` → `[selectedUser]`
   사용자가 바뀔 때마다 새 게시글을 가져와야 함
2. **조건부 조기 반환**: `if (!selectedUser) return;`
   selectedUser가 null일 때 fetch 방지

예상 결과
사용자 클릭 → Posts 탭에 게시글 목록 표시
다른 사용자 클릭 → 게시글 목록 교체

YOU DO — Task 3 & 4: 게시글 추가 / 삭제

Task 3 — POST 요청 | ★★★☆☆ | 20분

```
// addPost 함수 안에 구현 (필수 3)
const res = await fetch(`${API}/posts`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    title,
    body,
    userId: selectedUser!.id,  // ! = null 아님 보장
  }),
});
const newPost: Post = await res.json();
// id 충돌 방지: 서버는 항상 101을 반환하므로
setPosts((prev) => [{ ...newPost, id: Date.now() }, ...pre
```

Task 4 — DELETE 요청 | ★★☆☆☆ | 10분

```
// deletePost 함수 (필수 4)
const deletePost = async (id: number) => {
  await fetch(`${API}/posts/${id}`, {
    method: "DELETE",
  });
  // filter로 해당 id 제거 (불변 업데이트!)
  setPosts((prev) => prev.filter((p) => p.id !== id));
};
```

Task 4는 파일 두 개 수정!
`App.tsx` 의 deletePost 함수 구현 +
`PostCard.tsx` 의 버튼 onClick 연결 필요

YOU DO — Task 5 & 6: Todo 토글 / 댓글

Task 5 — Todo 완료 토글 | ★★★☆☆ | 20분
파일: `src/components/TodoSection.tsx`

핵심 패턴: map으로 특정 항목만 변경

```
// toggleTodo 함수 (필수 5-c)
setTodos((prev) ⇒
  prev.map((t) ⇒ {
    if (t.id ≠ id) return t;                // 해당 아님 → 
    return { ...t, completed: !t.completed }; // 해당 → com
  })
);
// PATCH 요청 (상태 업데이트 후)
fetch(`${API}/todos/${id}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ completed: true }),
});
```

Task 6 — 댓글 표시 | ★★★☆☆ | 20분
파일: `src/components/CommentList.tsx`

핵심 패턴: 처음 클릭 시에만 fetch

```
// toggle 함수 (필수 6-b)
const toggle = async () ⇒ {
  // 처음 열 때만 fetch
  if (!open && comments.length ≡ 0) {
    const res = await fetch(
      `${API}/posts/${postId}/comments`
    );
    const data: Comment[] = await res.json();
    setComments(data);
  }
  setOpen((prev) ⇒ !prev); // 토글
};
```

Task 5: 파일 아래쪽의 `const todos: Todo[] = []` 임시 변수 삭제 필수!

도전 과제 소개 (Tasks 7–11)

필수 과제 6개를 먼저 완성한 수강생을 위한 추가 도전입니다

Task	내용	난이도
7	AlbumGallery 구현	★★★☆☆
8	**Promise.all** 병렬 fetch	★★★★☆
9	**useMemo** 검색 필터링	★★★☆☆
10	Skeleton 로딩 상태	★★★★☆
11	StatsUpdateFn 콜백	★★★★★

Task 8 — Promise.all 핵심 개념

5개 API를 순서대로 호출하면 최대 2.5초

Promise.all로 병렬 호출하면 가장 느린 1개 분의 시간만!

```
Promise.all([
  fetch(`${API}/posts`).then(r => r.json()),
  fetch(`${API}/comments`).then(r => r.json()),
  // ...
]).then(([posts, comments, ...]) => {
  setStats({ posts: posts.length, ... });
});
```

트러블슈팅 Top 5

실습 중 가장 많이 나오는 오류들

① selectedUser is possibly null
useEffect 안 또는 함수에서 null 체크 없이 접근
→ `if (!selectedUser) return;` 추가

② Cannot redeclare block-scoped variable
useState를 만들었는데 임시 변수를 아직 삭제 안 함
→ App.tsx와 TodoSection.tsx의 임시 변수 삭제

③ 사용자 바꿔도 게시글 안 바뀜
의존성 배열에 `selectedUser` 미포함
→ `}, [selectedUser]);` 로 수정

④ useEffect async 직접 사용
`useEffect(async () => ...` 패턴 사용 시 경고
→ 내부에 async 함수 정의 후 호출 패턴으로 변경

⑤ 게시글 삭제/추가 후 목록 안 바뀜
`posts.push(newPost)` 같은 직접 변경
→ `setPosts(prev => [newPost, ...prev])`
또는 PostCard.tsx onClick 연결 누락

Quiz Time!

오늘 배운 내용을 확인합니다

문제 풀이 10분 · 해설 5분

오늘 배운 핵심 정리

오늘 배운 핵심 정리

TypeScript 핵심

- 기본 타입: string, number, boolean, 배열, 객체
- interface: 객체 구조 정의 (User, Post, Todo...)
- Union 타입: `User | null`, `string | number`
- 제네릭: `useState<User[]>`, `identity<T>`
- 옵셔널 체이닝: `user?.name`
- Non-null assertion: `user!.id`

오늘 배운 핵심 정리

TypeScript 핵심

- 기본 타입: string, number, boolean, 배열, 객체
- interface: 객체 구조 정의 (User, Post, Todo...)
- Union 타입: `User | null`, `string | number`
- 제네릭: `useState<User[]>`, `identity<T>`
- 옵셔널 체이닝: `user?.name`
- Non-null assertion: `user!.id`

React 핵심

- 컴포넌트: JSX를 반환하는 함수
- Props: 부모 → 자식 데이터 전달 (단방향)
- useState: 상태 관리, 변경 시 리렌더링
- useEffect: 사이드 이펙트 (fetch, 구독 등)
- 콜백 Props: 자식 → 부모 상태 변경 패턴
- 불변 업데이트: filter, map, 스프레드 연산자

오늘 배운 핵심 정리

TypeScript 핵심

- 기본 타입: string, number, boolean, 배열, 객체
- interface: 객체 구조 정의 (User, Post, Todo...)
- Union 타입: `User | null`, `string | number`
- 제네릭: `useState<User[]>`, `identity<T>`
- 옵셔널 체이닝: `user?.name`
- Non-null assertion: `user!.id`

React 핵심

- 컴포넌트: JSX를 반환하는 함수
- Props: 부모 → 자식 데이터 전달 (단방향)
- useState: 상태 관리, 변경 시 리렌더링
- useEffect: 사이드 이펙트 (fetch, 구독 등)
- 콜백 Props: 자식 → 부모 상태 변경 패턴
- 불변 업데이트: filter, map, 스프레드 연산자

오늘 실습에서 쓴 패턴 세 가지

`useState<T[]>([])` 제네릭 상태 선언 | useEffect + 내부 async 함수 패턴 | filter / map 불변 업데이트

더 공부하려면

다음 단계 라이브러리

- **React Query (TanStack Query)**
  서버 상태 관리, 캐싱, 로딩/에러 자동 처리
- **React Router**
  SPA 페이지 전환 (/home, /about 라우팅)
- **Zustand / Jotai**
  전역 상태 관리 — useState의 한계를 넘어
- **TypeScript Handbook**
  unknown vs any, Mapped Types, Conditional Types

공식 문서 링크

- React 공식 문서
  `react.dev`
- TypeScript Handbook
  `typescriptlang.org/docs/handbook`
- TypeScript Exercises
  `typescript-exercises.github.io`
- JSONPlaceholder API
  `jsonplaceholder.typicode.com`

React 공식 문서는 한국어 번역도 있습니다.
`ko.react.dev`

수고하셨습니다!

오늘 하루 TypeScript와 React의 핵심을
실제 앱으로 만들어 보셨습니다

참고 자료
react.dev
ko.react.dev
typescriptlang.org

실습 코드
solutions/ 폴더에서
정답 코드 확인 가능

Q&A
궁금한 점은
지금 질문해 주세요